

Received 28 October 2025, accepted 19 December 2025, date of publication 26 January 2026, date of current version 1 April 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3658096

 SURVEY

A Survey of Domain-Specific Fine-Tuned Large Language Models

PARIA SARZAEIM^{1,2}, AKRAMUL AZIM¹, (Senior Member, IEEE), GARY BAUER², AND MASOUD MAKREHCHI¹

¹Department of Electrical, Computer, and Software Engineering, Ontario Tech University, Oshawa, ON L1G 0C5, Canada

²Mobile Innovations Corporation, Niagara Falls, ON L2G 0M5, Canada

Corresponding author: Paria Sarzaeim (paria.sarzaeim@ontariotechu.net)

This work was supported in part by an industry partner, Mobile Innovations Corporation.

ABSTRACT The advancement of large language models (LLMs), such as GPT-3, BERT, and Llama, has introduced a new era in natural language processing (NLP) as they have demonstrated exceptional capabilities across diverse tasks. Fine-tuning these LLMs with domain-specific data has become a popular practice, particularly in domains like education, law, medicine, and software development. This process not only empowers LLMs with domain knowledge but also enhances their capabilities by addressing issues such as reliability and reducing the chance of hallucination. This survey examines a range of domain-specific fine-tuned models by highlighting their unique characteristics, enhancements in performance, and the challenges they address, including the demand for extensive computational resources. Furthermore, we explore the fine-tuning methodologies, including instruction tuning used in the development of these models, and ultimately, inform future research on the existing gaps and how to enhance the effective application of LLMs across diverse domains.

INDEX TERMS Fine-tuning, fine-tuned large language models, large language models, LLM.

I. INTRODUCTION

The advancement of large-scale language models has offered new opportunities and capabilities. LLMs leverage the power of over-parameterized transformers which can handle large-scale language or vision tasks with an impressive performance across various applications [1]. These improvements have revolutionized real-world applications across diverse domains, ranging from chatbots and writing assistants to search engines and multimodal systems. Notably, models like GPT-2 [2], GPT-3 [3], other GPT models, and Llama models [4] have marked a significant difference from previous LLMs. The impact of these models has raised widespread interest among both academic and industrial communities, and motivated investigations into their potential [5]. However, the general-purpose nature of these models often limits their performance when applied to specialized tasks. To address this limitation, fine-tuning LLMs on domain-specific data has emerged as a key strategy to enhance their capabilities in handling the unique language structure of specific fields.

The associate editor coordinating the review of this manuscript and approving it for publication was Genoveffa Tortora¹.

Domain-specific LLMs are LLMs customized to perform tasks in a specialized field, such as medicine, law, or finance. The workflow of creating domain-specific language models involves initial pretraining on extensive text corpora by employing unsupervised learning to establish a robust initialization for subsequent supervised learning on diverse downstream tasks [6]. This subsequent phase, known as fine-tuning, optimizes model parameters along with a small set of task-specific parameters.

The fine-tuning process provides LLMs with domain knowledge, enhances their specific capabilities, improves output fairness and reliability, and reduces issues such as hallucination [7], [8]. However, it presents challenges, such as the demand for substantial computational resources and data privacy concerns. Another fundamental challenge arises in the form of a misalignment between the training objectives of LLMs and the expectations of end-users. This disparity has prompted the exploration of fine-tuning techniques as a means of tailoring pretrained models to specific tasks and user needs [7].

Fine-tuning techniques have gained prominence in addressing this challenge, particularly with instruction tuning

as an approach designed to enhance controllability and conversational capabilities. By further training LLMs with pairs of instructions and outputs, this method not only aligns models with user expectations but also offers efficient computational means to adapt LLMs to specific domains without extensive training [7]. Despite the effectiveness of fine-tuning, challenges persist, ranging from the crafting of high-quality instructions to concerns regarding the effectiveness of the method on tasks heavily supported in the training dataset. Additionally, adapting large-scale pretrained LLMs for various downstream applications through fine-tuning presents a challenge because of the computational expense associated with updating all model parameters [9].

This survey aims to explore domain-specific fine-tuned LLMs and review prevalent models and methodologies. Through an examination of the existing literature and empirical experiments, we aim to provide a framework for understanding the complex interplay between general-purpose LLMs and their fine-tuned variants, as well as to contribute to the ongoing research about the optimization of these models for real-world applications and propose future directions for research and development. To this end, the remainder of this paper is organized as follows. Section II provides an overview of related work, along with background information on fine-tuning and common LLMs. Section III lists the domain-specific fine-tuned LLMs in different fields and applications, and Section IV highlights the research findings and the existing gaps in the literature. Finally, Section V concludes the paper and highlights potential avenues for future work.

II. BACKGROUND

There are several related works in the field of fine-tuning LLMs for domain-specific applications. However, our work distinguishes itself through its broader scope and comprehensive coverage of various applications. While previous works have focused on specific domains or tasks, this paper provides a general overview of fine-tuning LLMs across a diverse range of domains, including finance, medicine, legal applications, and programming. Researchers in [10] focus on fine-tuning LLMs specifically for machine translation tasks, which narrows the scope to translation-related tasks in specialized domains. Similarly, works like [11] emphasize specific techniques for fine-tuning with high-quality domain-specific data to prevent capability collapse in domain adaptation. While the methodologies in these related works focus on prompt-oriented fine-tuning or quality assessment frameworks, this paper highlights the importance of supervised learning and instruction tuning as key approaches for improving performance in domain-specific tasks. Furthermore, some related works discuss niche applications, such as rare disease normalization [12] or robot programming [13], which involve fine-tuning models on specialized, often narrow, datasets. Finally, the work presented in [14] explores training strategies, model merging, and scaling for domain adaptation.

As we navigate through our exploration of domain-specific fine-tuned LLMs, it is crucial to highlight the motivation behind our survey and the preliminary information. pre-training is an unsupervised learning procedure that equips LLMs with robust initialization and groundwork for further supervised learning on downstream tasks through fine-tuning. This additional optimization involves adjusting the model parameters along with a small set of task-specific parameters [15].

The fine-tuning of LLMs is a method to align these models more closely with user needs. Training LLMs, which often focus on next-word prediction, may not fully answer user expectations for precise and helpful responses. Instruction tuning has emerged as a technique to address this discrepancy and provide an effective channel for humans to guide and intervene in a model's behavior. While instruction tuning has improved task alignment in LLMs, recent analysis reveals key limitations e.g., instruction tuning can degrade core model knowledge, introduce copying behavior from instruction datasets, and increase hallucinations for out-of-distribution inputs [16]. However, it remains a widely used method for domain-specific fine-tuning of LLMs. With this background, the purpose of this survey is to review the applications of fine-tuning LLMs and understand the current complexities and methodologies.

Although our focus is on domain-specific fine-tuned LLMs, it is noteworthy to mention techniques that reduce computational challenges during this process. Adapting LLMs for various downstream applications through fine-tuning presents a challenge of the computational expense associated with updating all the model parameters [17]. The growth in model size like GPT-3 [3] with 175 billion parameters, has turned minor computational issues for smaller models like GPT-2 [2] into major deployment challenges.

To address these challenges, new methods, such as using adapter layers and optimizing input-layer activations, are used. Adapter layers, while designed with a small bottleneck dimension to limit added FLOPs [18], [19], introduce latency, particularly in online inference settings with small batch sizes [20], [21]. Prefix tuning also offers a lightweight alternative to full fine-tuning by only optimizing a small continuous token sequence. Optimizing input layers, as in prefix tuning, faces challenges in terms of non-monotonic performance changes and reduced sequence length availability for downstream tasks [22]. In response to these issues, the Low-Rank Adaptation (LoRA) approach is proposed, which uses the low intrinsic dimension of overparameterized models during adaptation. LoRA optimizes rank decomposition matrices for dense layer changes during adaptation and enables efficient training of dense layers while keeping pretrained weights frozen [17]. However, despite its parameter efficiency, LoRA presents some practical limitations. Recent research has shown that LoRA's computational advantages can degrade sharply when input or gradient norms exceed specific thresholds, which

leads to suboptimal scaling in long-sequence or multi-head attention settings [23]. Additionally, LoRA's performance is also highly sensitive to adapter rank and layer selection, and it requires careful tuning to avoid underfitting or overfitting, particularly in low-resource domains. Furthermore, integrating LoRA into multi-modal or quantized inference pipelines introduces additional deployment challenges due to architectural mismatches or incompatibilities with latency-sensitive environments [24].

GOBO model quantization technique is also proposed [25], which compresses the majority of 32-bit floating-point parameters in BERT models to 3 bits without requiring fine-tuning or retraining, thereby reducing memory storage, traffic, and inference latency. A previous paper introduced the Int8 matrix multiplication procedure for feed-forward and attention projection layers in transformers, cutting the memory needed for inference by half while maintaining full precision performance [26]. Furthermore, SLIMFIT dynamically analyzes training dynamics and freezes less-contributory layers during fine-tuning, adopting quantization and pruning to reduce memory requirements for transformer-based models such as BERT [27].

These methods can be used to reduce computational resources in the process of domain-specific fine-tuning LLMs. In this section, we list LLMs that are widely used as the base model in this field.

A. GPT-3

GPT-3, or the Generative pretrained Transformer 3, is a state-of-the-art that belongs to the transformer architecture family as shown in Figure 1 from [28] and suggests a significant improvement in comparison with previous models that relied on recurrent or convolutional layers to process sequential data. GPT-3 uses the same model as GPT-2, with alternating dense and sparse attention patterns. The main innovation of GPT-3 lies in the utilization of the attention mechanism, enabling the model to weigh the importance of different words in the input data relative to each other and capture language syntax and semantics more effectively [25].

As shown in Figure 2, the GPT-3 has a layered structure with a decoder component in the transformer architecture. This design reflects the focus of the model on generating text; therefore, it uses an input sequence of tokens and predicts the next token in the sequence in an autoregressive manner. The embedding layer converts each input token into a high-dimensional vector. This vector representation allows the model to capture semantic similarities between words as tokens that are semantically related are mapped to vectors that are close to each other in the embedding space. After scaling the embedding vectors, positional encoding is applied to add information regarding the order of tokens in the sequence by adding signals to these vectors that indicate the position of each token within the sequence. Following these initial encoding steps, the input data are processed through a series of decoding layers. Each layer in the decoder, as shown in Figure 3, is designed to refine the representation of the

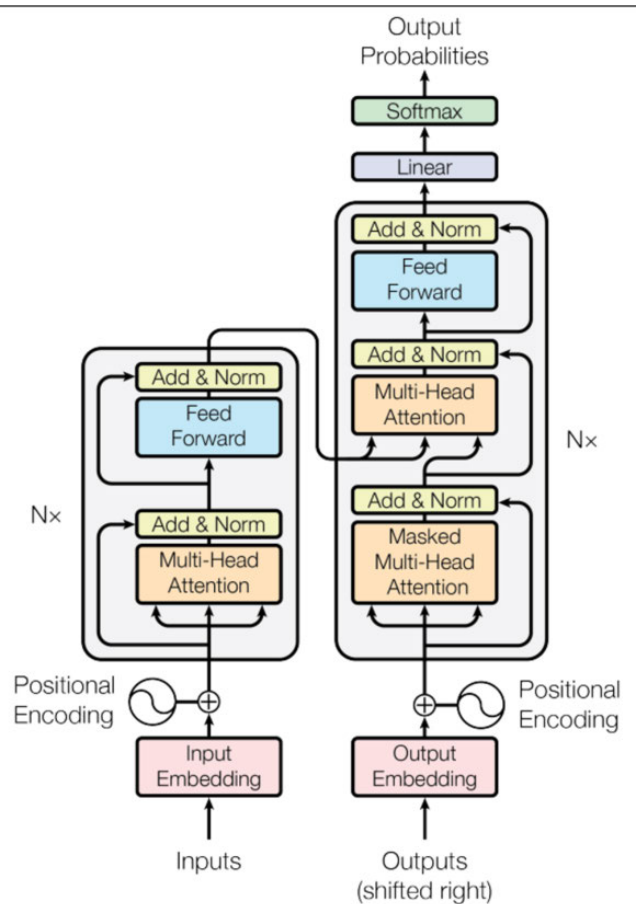


FIGURE 1. The transformer model architecture (adopted from [28]).

input data by focusing on different aspects of the information contained in the embedding vectors. These layers use the attention mechanism to compute self-attention scores, which determine how much focus should be placed on each token when predicting the next token. These scores are calculated through a series of dot product operations between the query, key, and value vectors derived from the tokens themselves. Moreover, GPT-3 employs multiheaded attention to segment the attention mechanism into multiple heads that operate in parallel, which allows the model to capture a variety of linguistic features and relationships at different positions within the input sequence. The output from the decoder layers is then normalized to ensure that the embeddings are scaled and passed through a linear layer, which adjusts the dimensionality of the data to match the size of the model's vocabulary. Finally, a softmax layer converts these predictions into a probability distribution over vocabulary. This distribution represents the model's predictions for the next token to generate text by selecting tokens with the highest probability scores.

The training dataset of GPT-3 was created by filtering CommonCrawl [29], and adding reference corpora. GPT-3 exhibits impressive language understanding, generation, and completion capabilities. As the model has 175 billion parameters and is pretrained on a broad range of text datasets

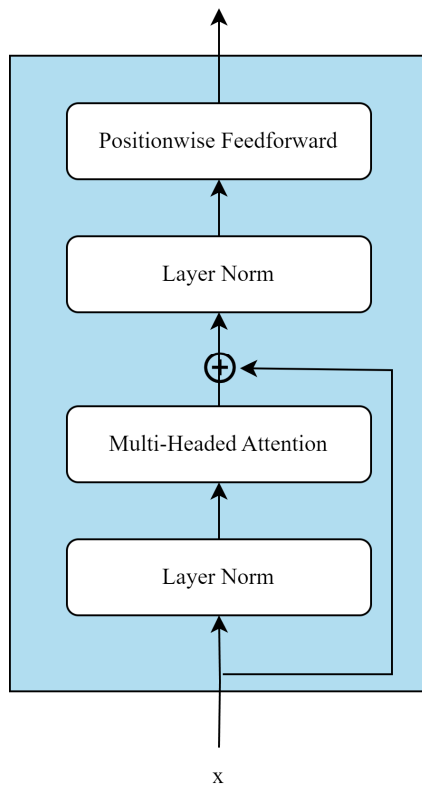


FIGURE 2. Structure of a decoder layer in GPT-3.

with 300 billion tokens to generate human-like text, it is commonly used in domain-specific fine-tuning.

B. GPT-4

GPT-4, with the same transformer architecture as GPT-3, was introduced in March 2023. Compared to its predecessor, GPT-3.5, GPT-4 exhibits enhanced problem-solving abilities, resulting in substantial performance improvements across various evaluation tasks. A recent study conducted qualitative tests and demonstrated GPT-4's superior performance over prior GPT models in diverse and challenging tasks [30].

GPT-4 demonstrated improved safety in responding to malicious queries, attributed to a six-month iterative alignment and the incorporation of a safety reward signal in the Reinforcement Learning with Human Feedback (RLHF) training. OpenAI's technical report emphasizes safe development practices for GPT-4 by addressing concerns such as hallucinations and overreliance. Additionally, intervention strategies, including red teaming, have been introduced to mitigate the potential risks associated with LLMs. GPT-4 was built on a deep learning model in addition to optimization methods. According to these improvements, GPT-4 is another powerful candidate as the base model in domain-specific fine-tuning.

C. BERT

BERT, or Bidirectional Encoder Representations from Transformers introduced by researchers at Google, is a language

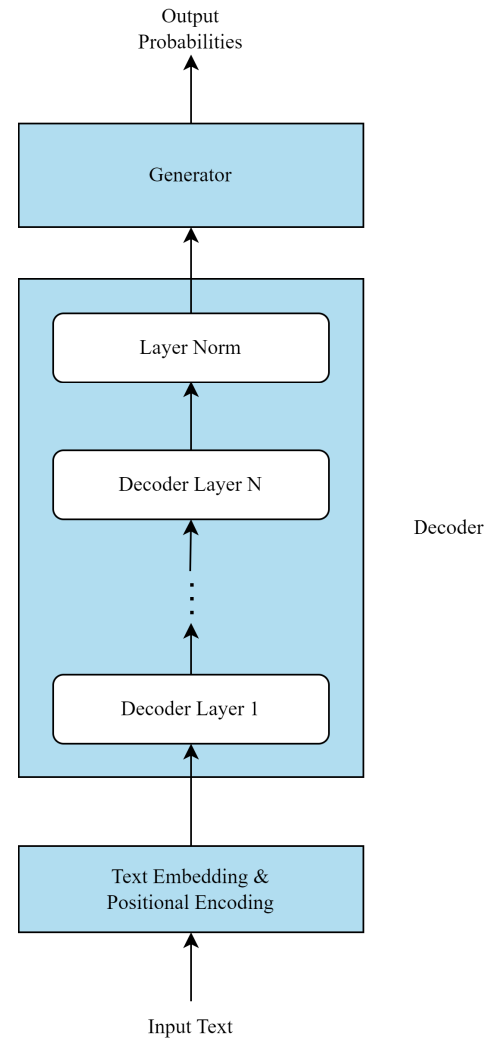


FIGURE 3. GPT-3 layered structure.

representation model to pretrain deep bidirectional representations from unlabeled text. The model is conceptually simple yet powerful and has achieved remarkable results on 11 natural language processing tasks, including question-answering, language inference, and token-level tasks, such as named entity recognition [31].

The key innovation of BERT lies in its bidirectional pretraining approach, which overcomes the limitations of unidirectional language models. Traditional models that process text in a linear sequence or autoregressive models, such as GPT, which predict each subsequent word while masking previous ones, are either left-to-right or right-to-left, restricting their ability to capture context efficiently for tasks requiring bidirectional understanding. BERT introduces a masked language model (MLM) objective, where a random percentage of input tokens are masked, and the model predicts the original vocabulary ID of the masked word based on its context. This enables BERT to fuse the left and right contexts and create deep bidirectional representations.

The model comprises two steps: pretraining on unlabeled data and fine-tuning of labeled data for specific tasks which involves leveraging the bidirectional self-attention mechanism in the transformer for various downstream tasks. BERT's unified architecture and bidirectional pretraining of BERT significantly reduce the need for task-specific architecture engineering [31].

BERT was released in several variations, including Base and Large models, with options for processing text in both cased and uncased forms. Therefore, BERT can be adapted to various languages and specialized tasks. DistilBERT was also introduced to condense the power of BERT into a smaller, more efficient package. This distilled version of BERT maintains much of the original model's language understanding capabilities, but is significantly smaller and faster; therefore, it is more practical for real-world applications where computational resources are limited. The distillation process involved training DistilBERT to replicate the output probabilities of the BERT model and to incorporate the original MLM objective. Additionally, cosine embedding loss ensures that DistilBERT's hidden states are close to those of BERT which helps it to learn a representation similar to that of the English language. DistilBERT was pretrained in a self-supervised manner on the same corpus using the BERT base model [32].

D. T5

The T5 or Text-To-Text Transfer Transformer is a specific type of language model based on the transformer architecture developed by Google Research [33]. T5 is part of the transformer family of models and is designed to convert all tasks into a text-to-text format and treat both the inputs and outputs as sequences of text. This formulation allows the model to be pretrained and fine-tuned on a diverse set of tasks using the same architecture and training objective. The architecture of T5 follows a standard transformer design by employing a bidirectional encoder-decoder structure. In this framework, the encoder processes the input sequence and captures the bidirectional context through self-attention, whereas the decoder generates the output sequence autoregressively. A key feature of T5's pretraining is its use of a variant of the MLM known as the "span-corruption" objective. Instead of masking individual tokens, T5 masks contiguous spans of text and replaces them with a special token. The pretraining has been done on a large and diverse corpus, which included publicly available datasets, such as the Colossal Clean Crawled Corpus (C4) [33]. This large-scale pretraining allows the model to learn general language representations that can be fine-tuned for specific tasks. The study behind T5 systematically compared different architectural variants, pretraining objectives, and unlabeled datasets, and revealed that the choice of dataset and the pretraining task can significantly affect the performance of the model on downstream tasks. In terms of performance, T5 achieved state-of-the-art results on numerous benchmarks, including summarization, question-answering, and classification [33].

Flan-T5, or Fine-tuned Language Net T5, is an extension of the original T5 model, aimed at improving generalization on unseen tasks through instruction-based fine-tuning. The key difference between T5 and Flan-T5 lies in how Flan-T5 is trained on a wide variety of instruction tuning datasets, which enables it to perform better zero-shot and few-shot learning tasks without requiring task-specific fine-tuning. The Flan collection of datasets on which Flan-T5 is trained on integrates a diverse set of tasks and methods, that improved its performance over previous models, such as T5, and even larger models, such as PaLM, on specific evaluation suites. For example, Flan-T5 outperforms other models by 3-17% on benchmarks such as Massive Multitask Language Understanding and Big-Bench Hard [34].

E. LLAMA

Llama models were introduced by Meta in February 2023 and have received significant attention in the realm of LLMs and stand out as leading open-language models. Llama extracts training data from various sources, including CommonCrawl [25], C4 [33], GitHub, Wikipedia, Books3 [35], ArXiv, and StackExchange. The training data sizes for Llama 6B and Llama 13B were 1.0T tokens, while 1.4T tokens were used for Llama 32B and Llama 65B [4]. Researchers have extended these models through instruction tuning or continual pretraining, with instruction tuning becoming a prominent method for developing customized models at a relatively low computational cost. Adapting Llama to non-English languages often involves expanding the original English-trained vocabulary or fine-tuning it with instructions or data in the target language. Stanford Alpaca, a notable Llama variant, is the first open instruct-following model fine-tuned on Llama 7B, leveraging 52K instruction-following [36]. This model has inspired subsequent works, such as AlpacaLoRA, Koala, and BELLE. Vicuna is also another variant of Llama trained on user-shared conversations from ShareGPT [37]. Leveraging the performance of the Llama family, multimodal models integrate them as base language models to enhance their capabilities. In particular, Vicuna is preferred in multimodal language models and is used in the development of models such as LLaVA and PandaGPT [4].

III. DOMAIN-SPECIFIC FINE-TUNING

The models included in this survey were identified primarily through searches on Google Scholar and the IEEE Xplore digital library. We used combinations of keywords related to large language models and domain specialization, such as "large language models," "LLM fine-tuning," "instruction tuning," and domain-specific terms including e.g., "medical," "medicine," and "healthcare" for medical application. We restricted our selection to papers written in English. Candidate papers were initially assessed based on their titles, keywords, and abstracts, followed by a brief examination of the main content to verify relevance. Models were included if their paper clearly described a fine-tuning or adaptation

method applied to an LLM for a specific domain or task, provided empirical evaluation results, and discussed benefits, limitations, or practical challenges.

Based on this selection, the surveyed domain-specific fine-tuned LLMs are categorized into seven application groups: Financial, medical, writing, education, agriculture, application, and software development. Tables 1-7 presents a brief overview of these models and information about their base model, the dataset used in their fine-tuning, the required computational resources, and their evaluation metrics or frameworks.

A. FINANCIAL

There are many fine-tuned LLMs in the financial sector as listed in Table 1. The earlier versions of fine-tuned LLMs are based on classic supervised fine-tuning. FinBERT is built on pre-trained BERT-base and BERT-large architectures and is adapted to the financial domain via further pre-training on TRC2-financial, a corpus of Reuters financial news articles, followed by fine-tuning on labeled financial sentiment datasets such as Financial PhraseBank and FiQA. [38]. FinBERT outperformed traditional sentiment analysis methods, such as the Loughran-McDonald dictionary and machine learning classifiers like SVM, CNN, and LSTM by demonstrating higher accuracy, particularly in distinguishing negative sentiment. The model uses techniques such as slanted triangular learning rates, discriminative fine-tuning, and gradual unfreezing to stabilize training and improve performance even with limited labeled data. This means that instead of applying a uniform learning rate across the entire model, FinBERT assigns higher learning rates to later layers that are more task-specific, and lower learning rates to earlier layers that capture general language features to prevent catastrophic forgetting, which refers to the phenomenon where a model, when fine-tuned on a new domain or task, may lose or degrade previously learned general knowledge and reasoning abilities [45]. Beyond sentiment analysis, FinBERT has been extended to classify discussions related to environmental, social, and governance issues, which demonstrates its adaptability in financial text processing. However, it struggles with more complex financial reasoning, such as market prediction or risk analysis. Additionally, as an encoder-only model trained without instruction tuning, it lacks instruction-tuned capabilities.

PIXIU is a framework that addresses crucial gaps in financial artificial intelligence and presents FinMA, the first financial LLM fine-tuned from Llama [41]. The fine-tuning process uses the FIT (Financial Instruction Tuning) dataset, a multi-task and multi-modal dataset containing 136,609 samples that covers a broad range of financial tasks, including sentiment analysis, news classification, named entity recognition, question answering, and stock movement prediction. First, financial text data is preprocessed into instruction-based formats. FinMA is then built by fine-tuning Llama-7B and Llama-30B, as well as a FinMA-7B-full variant trained on both NLP and prediction tasks. Training

uses the AdamW optimizer with a learning rate of 8×10^{-6} , and parameter-efficient adaptation via LoRA. Training is conducted on 8 A100 GPUs for FinMA-7B and 128 A100 GPUs for FinMA-30B, with up to 20 epochs, depending on model size. Evaluation is conducted using FLARE, a standardized benchmark comprising four financial NLP tasks and one financial prediction task across nine datasets. The results show that FinMA achieves strong and often superior performance compared to BloombergGPT and GPT-4 on financial sentiment analysis and NER under zero-shot evaluation. However, it underperforms on question answering and exhibits only limited gains on stock movement prediction which indicates the need for further enhancements in financial numerical reasoning.

Instruct-FinGPT is another model specific to financial sentiment analysis that utilizes instruction tuning to enhance LLM performance [6]. Instruct-FinGPT is built upon the Llama 7B model to address two major challenges in financial NLP: numerical sensitivity and contextual comprehension. In the fine-tuning process, first, financial sentiment classification datasets, including the Twitter Financial News Sentiment dataset and the FiQA dataset are reformatted into instruction-based prompts. Each data sample is paired with a human-like instruction to transform the task from a standard classification problem into a sequence-to-sequence generation task. Next, the instruction-formatted dataset is used to fine-tune Llama-7B and AdamW optimizer with a learning rate of 1×10^{-5} , weight decay of 0.1, and a batch size of 32. The model undergoes 10 epochs of instruction tuning on 8 A100 40GB GPUs using DeepSpeed to optimize memory usage and parallel computation. Finally, since Llama-7B is a decoder-only model, its generated responses are post-processed and mapped back to sentiment labels. This fine-tuning strategy significantly enhances the model's zero-shot financial sentiment classification ability. While the datasets used in the process of fine-tuning Instruct-FinGPT are small and may cause low generalization, it requires high computational resources.

FinGPT is a framework designed to adapt LLMs to financial domain tasks and address restricted data access, lack of transparency, and the proprietary nature of existing financial language models such as BloombergGPT [42]. The framework automates the collection of real-time financial data from more than 34 heterogeneous sources, including news platforms, social media, financial reports, and stock market data. These data are processed through cleaning and filtering pipelines to mitigate the low signal-to-noise ratio typical of financial text. FinGPT introduces Reinforcement Learning with Stock Prices (RLSP), in which stock price movements are used as proxy supervision signals for sentiment classification. This approach automates label generation and supports frequent model re-adaptation, although the training itself follows a supervised fine-tuning procedure rather than classical reinforcement learning. To reduce adaptation costs, the framework relies on lightweight fine-tuning methods such as LoRA and QLoRA.

TABLE 1. Fine-tuned models for financial applications.

Fine-tuned Model	Base Model	Dataset	Dataset Size	Model Size	Computational Resources	Evaluation Metric/Framework
FinBERT [38]	BERT-base and BERT-large [31]	TRC2-financial (Reuters) [38]	46,143 news articles	110M/340M	Amazon p2.xlarge EC2 instance with one NVIDIA K80 GPU, 4 vCPUs, 64 GiB RAM	R^2 (regression), Cross Entropy Loss, Macro F1, and Accuracy
		Financial Phrase-Bank dataset [39]	4,845 English sentences			
		FiQA [40]	1,174 financial news and tweets			
FinMA [41]	Llama variants [4]	FIT [41]	136,609 samples	7B/30B	8 A100 40GB GPUs for FinMA-7B and 128 A100 40GB GPUs for FinMA-30B	Accuracy, F1 Score, AUC-ROC (assessed through FLARE benchmark tasks)
Instruct-FinGPT [6]	Llama 7B [4]	FiQA [40]	961 samples	7B	8 A100 40GB GPUs	Accuracy, F1 Score, Precision, Recall
		Twitter financial news	9,540 samples			
FinGPT [42]	Llama 2 [4], Chat-GLM2 [43], and ChatGPT [44]	Financial data from online sources on news, social media, and trends	More than 34 sources are used, but their size is not specified	Not specified	A100 and RTX3090 depending on the base model	Accuracy, F1 Score, and Cumulative Return Rate in the context of trading experiments

FinGPT is evaluated using base models, including Llama 2, ChatGLM2, and ChatGPT through three demonstrative financial applications including robo-advisory, sentiment analysis for quantitative trading, and low-code financial development. The robo-advisor and low-code development applications are presented qualitatively through case studies and prompt-based examples, demonstrating FinGPT's ability to generate financial analyses, summaries, and code artifacts. Quantitative evaluation is primarily conducted in the context of sentiment analysis for quantitative trading. Experiments are focused on two fine-tuning settings:

- Market-feedback-driven labeling via RLSP where financial news related to S&P 500 constituents is automatically labeled using subsequent stock price movements with a 2% threshold. Performance is evaluated using accuracy and macro-F1 and a simulated trading strategy, where sentiment predictions generate predefined buy or short actions. Results show that FinGPT-fine-tuned models consistently outperform untuned LLM baselines, with the Llama 2 model achieving an F1 score of 0.894 and a 9.6% improvement in average cumulative return rate.
- Standard supervised fine-tuning using established financial sentiment datasets where FinGPT demonstrates consistent performance gains across all evaluated datasets and base models.

Financial LLMs exhibit diverse fine-tuning strategies tailored to the unique challenges of financial language,

including numerical precision, sentiment volatility, and real-time adaptability. Early models like FinBERT rely on supervised fine-tuning and learning rate scheduling to adapt general LMs for sentiment classification using limited labeled data, while newer models such as FinMA, Instruct-FinGPT, and FinGPT introduce instruction tuning, LoRA-based parameter efficiency, and even reinforcement learning from market feedback. FinMA uses a multi-task Financial Instruction Tuning dataset to train models across tasks like classification and prediction, whereas Instruct-FinGPT converts classification tasks into generative sequences to optimize for zero-shot generalization in constrained settings. FinGPT extends this further by introducing real-time data collection, dynamic sentiment labeling using stock price movements, and reward-based tuning, which offers high adaptability but also raises concerns around reliability and misinformation. These evolving fine-tuning strategies reflect a shift from static task-specific models to adaptable, real-time systems, driven by financial applications' demand for interpretability, risk sensitivity, and decision-grade accuracy.

B. MEDICAL

Researchers have designed and fine-tuned LLMs for many medical applications as listed in Table 2. BioBERT is a language model that is specifically designed for biomedical domains [46]. BioBERT follows a classic supervised fine-tuning approach after domain-specific pretraining. First,

it is initialized with the BERT-base model, which was pretrained on English Wikipedia and BooksCorpus. It is then further pretrained on biomedical corpora (PubMed and PMC) using masked language modeling and next-sentence prediction objectives to capture the specialized vocabulary, terminology, and contextual nuances of biomedical literature. After pretraining, BioBERT undergoes supervised fine-tuning on downstream biomedical NLP tasks using labeled datasets. These tasks include biomedical named entity recognition, relation extraction, and question-answering, where BioBERT consistently outperforms general-domain BERT models by better capturing biomedical terminology and context.

SciFive is a domain-adapted variant of the T5 designed for biomedical natural language processing (BioNLP) tasks [47]. Unlike BERT-based models that operate on fixed classification outputs, SciFive adopts a sequence-to-sequence architecture to handle complex text generation tasks, such as document summarization, question answering, and biomedical entity relation extraction. SciFive follows a two-stage adaptation process, consisting of continued self-supervised pretraining on biomedical corpora followed by supervised fine-tuning on task-specific datasets. The model is initialized with T5 pretrained weights on a combination of general, biomedical, and clinical datasets using span-based masked language modeling, where random spans of text are masked, and the model learns to predict them in a structured output format. The model is then fine-tuned on multiple biomedical NLP tasks, including named entity recognition (disease, chemical, genes, and species), relation extraction (e.g., protein-chemical), natural language inference, document classification, and question answering. Both BioBERT and SciFive are two of the earliest biomedical NLP models. BioBERT follows the classic supervised fine-tuning, while SciFive follows a text-to-text paradigm; they do not have explicit instruction tuning, which is the case for newer LLMs.

DoctorGLM is a bilingual fine-tuned LLM based on ChatGLM-6B, developed to enhance medical question-answering and dialogue systems in both Chinese and English [48]. Unlike general-purpose models such as ChatGPT and GPT-4, which often demonstrate suboptimal precision in medical applications, DoctorGLM is specifically trained on medical dialogues and clinical interactions to improve diagnostic accuracy, drug recommendations, and patient consultations. The model is first fine-tuned using a curated dataset of Chinese medical dialogues, collected and synthesized with ChatGPT-generated interactions. This dataset includes real-world patient descriptions, diagnoses, and treatment suggestions. An important contribution of DoctorGLM is demonstrating that domain-specific medical LLMs can be trained and deployed at very low cost. LoRA and P-Tuning v2 were evaluated for this purpose, and ultimately LoRA was selected due to its ability to fine-tune the model using only seven million additional trainable parameters. This optimization enabled DoctorGLM to be

trained on a single NVIDIA A100 80G GPU, using a batch size of four and a maximum sequence length of 512 tokens. DoctorGLM also inherits alignment from ChatGLM-6B, which was trained with supervised fine-tuning, feedback bootstrapping, and RLHF. One key feature of the model is its prompt designer module, which extracts relevant disease-related keywords from user inputs and generates professional disease descriptions using a comprehensive medical knowledge library. Instead of using standard NLP benchmarks such as MedQA or USMLE, the model's performance was assessed through qualitative comparisons with ChatGPT across various real-world medical queries. DoctorGLM demonstrated higher specificity in diagnosis and drug recommendations, though it exhibited a tendency to provide overly definitive answers, whereas ChatGPT remained more conservative. Additionally, expert evaluations identified cases where DoctorGLM made factually incorrect diagnoses, which indicates the risk of hallucinations.

Radiology-GPT is a specialized LLM for radiology, to address the scarcity of models tailored to medical domains [49]. Radiology-GPT is designed for radiology text interpretation and clinically styled impression generation. Built on the Alpaca 7B model, an instruction-tuned variant of Llama 7B, it is fine-tuned using MIMIC-CXR, a dataset with over 60,000 chest X-ray reports. The model is first trained on structured radiology reports, particularly on Findings–Impression pairs, to learn diagnostically relevant language patterns. Then, using LoRA, the model is fine-tuned to generate concise, coherent, and clinically useful diagnostic impressions from raw findings. Evaluation against general-purpose LLMs such as ChatGPT, StableLM, and Dolly-12B demonstrates competitive or superior performance in conciseness and clinical reporting. The authors emphasize local deployment for privacy preservation and Health Insurance Portability and Accountability Act (HIPAA) regulations. Consequently, the authors highlight the importance of existing bias in the dataset, fairness, accountability, and transparency in model training, along with careful regulatory oversight and interpretability, as prerequisites for responsible clinical adoption. It is also noteworthy to mention that Radiology-GPT may exhibit domain-specific biases, as its training is primarily based on chest X-ray datasets and limits its adaptability to other imaging modalities such as MRI and CT scans.

Med42-v2 is a group of clinical LLMs built on Llama 3 and Llama 3.1 architectures [51]. The model is available in 8-billion and 70-billion parameter configurations and is fine-tuned in a two-stage process to enhance medical reasoning and response generation. The first stage involved clinical fine-tuning using a dataset with 73.5% medical data and 26.5% general-domain data that includes clinical question-answering, conversational examples, and chain-of-thought reasoning to gain broad linguistic and contextual adaptability for healthcare applications. The training utilized concatenated samples of up to 8192 tokens. The fine-tuning process

TABLE 2. Fine-tuned models for medical applications.

Fine-tuned Model	Base Model	Dataset	Dataset Size	Model Size	Computational Resources	Evaluation Metric/Framework
BioBERT [46]	BERT-base [31]	PubMed abstracts	4.5B words	110M	8 NVIDIA V100 GPUs for pretraining and single NVIDIA Titan Xp (12GB) GPU for fine-tuning	Precision, Recall, F1 Score
		PMC full-text articles	13.5B words			
SciFive [47]	T5-based and T5-Large [33]	PubMed abstracts and PMC and multiple task-specific biomedical datasets for fine-tuning	-	220M/770M	TPUv2-8 (Google Colab)	Precision, Recall, F1 Score, Lenient accuracy
DoctorGLM [48]	ChatGLM [43]	CMD	882K	6.2B	A100 80G GPU	General comparison/No specific metric
		MedDialog	3.4M			
		ChatDoctor	5.4K			
		HealthcareMagic	200K			
Radiology-GPT [49]	Alpaca 7B [36]	MIMIC-CXR dataset [50]	124,577	7B	4 NVIDIA A100 80GB GPUs	Understandability, Coherence, Relevance, Conciseness, Clinical Utility
Med42-v2 [51]	Llama3 and Llama3.1 [4]	Various sources (medical/general)	~ 1.3M	8B/70B	2 × 8 H100 GPUs (Llama 8B), up to 6 × 8 H100 GPUs (Llama 70B)	Zero-shot evaluation on medical benchmarks (Eleuther AI's harness)
ChatDoctor [52]	Llama [4]	Alpaca dataset [36]	52K	7B	6 A100 GPUs	BERTScore (Precision, Recall, F1)
		HealthCareMagic [53]	100K			
HuatuoGPT-II [54]	Baichuan2 variants	Huatuo-26M and Large-scale medical corpora	1.11T raw corpus	7B/13B	Multiple NVIDIA A100 cards	Medical question answering benchmarks and national licensing exams
Me-Llama [55]	Llama 2 variants [4]	Samples from MedInstruct, WikiDoc, Medical Flashcards, and other medical question answering datasets	129B tokens for pretraining and 214K samples for fine-tuning	13B/70B	8 A100 GPUs for 3 epochs	Medical Evaluation Benchmark, Macro-F1 Score, Accuracy, Exact Match, ROUGE-L, and BERTScore
DiabeTalk [56]	WizardLM [57]	Synthetic patient profiles (expert-labeled)	200 samples	Not specified	N/A (inference only)	Accuracy, Precision, Recall, F1-score, AUC

spans two epochs that use an autoregressive loss function with AdamW optimization, a linear warm-up phase, and a cosine decay learning rate schedule for improved convergence. The second stage, preference alignment, utilized datasets like UltraFeedback and Snorkel-DPO to refine outputs for safety

and relevance using Direct Preference Optimization, which is a technique used to align the outputs of a model with human preferences by directly optimizing a preference-based loss function as an alternative to reinforcement learning with human feedback. Med42-v2 was evaluated using zero-shot

prompting on medical benchmarks such as MMLU, USMLE, and MedQA and has demonstrated substantial improvements. The 8B model achieved an average score of %68.1, while the 70B model scored %81.2 and outperformed GPT-4. These advancements highlight the effectiveness of targeted fine-tuning and alignment. However, challenges like high computational resources, hallucinations, and biases persist, suggesting further research for safe real-world applications.

ChatDoctor [52] is another fine-tuned medical LLM based on the Llama 7B model. The model is instruction fine-tuned on Stanford's Alpaca dataset with 52K instruction-following prompts to develop general conversational capabilities and further fine-tuned on the HealthCareMagic-100k patient-doctor dialogue dataset. This augmentation substantially enhances the capacity of the model to understand patient requirements and provide more informed or contextually relevant responses. To further enhance accuracy and credibility and support response generation, ChatDoctor is equipped with a retrieval-augmented generation (RAG) mechanism to autonomously retrieve relevant information from external medical knowledge sources before generating responses. ChatDoctor was evaluated using real-world patient queries and benchmarked against ChatGPT using the iCliniq dataset, where human physician responses served as the gold standard. The model outperformed ChatGPT on semantic similarity metrics and could also handle recent medical topics, such as Monkeypox and newly approved medications, due to its external knowledge retrieval capability. Nevertheless, ChatDoctor remains a research prototype, with limitations related to bias, hallucinations, safety, and the absence of clinical validation.

HuatuoGPT-II is a fine-tuned LLM for the Chinese medical domain, developed to address the limitations of two-stage domain adaptation approaches [54]. Traditional models undergo separate unsupervised pretraining and instruction-based fine-tuning, which may lead to catastrophic forgetting and optimization difficulties; however, HuatuoGPT-II converts all pretraining and fine-tuning data into a unified instruction-output format. The model is built upon Baichuan2-7B and Baichuan2-13B base models and is trained on two key types of corpora: *structured medical literature and guidelines*, including medical literature (Chinese and English), textbooks, encyclopedias, and real-world patient-doctor questions and answers, derived from Huatuo-26M, a large-scale medical question answering dataset. To bridge the gap between these heterogeneous data sources, the training pipeline reformats all text into a standardized (instruction, output) pair format to simultaneously learn general medical knowledge and instruction-following capabilities. Additionally, the priority sampling strategy further enhances HuatuoGPT-II's training efficiency. As training progresses, the data distribution shifts toward real-world medical dialogues to have a balanced optimization between medical accuracy and conversational fluency. HuatuoGPT-II is trained using multiple NVIDIA A100 GPUs, employing the AdamW optimizer with a cosine decay learning rate

schedule to stabilize convergence. The model supports a maximum sequence length of 8192 tokens to facilitate the integration of long-form medical texts and complex multi-turn dialogues. HuatuoGPT-II is benchmarked against GPT-4, ChatGPT, Baichuan2, and ChatGLM, across Chinese medical licensing examinations (Pharmacy and Traditional Chinese Medicine) and popular medical NLP benchmarks (MedQA, MedMCQA, CMExam). The HuatuoGPT-II 13B model achieves a 52.9% score on the 2023 Pharmacist Licensing Exam, closely matching GPT-4's 57.3% score. Additionally, expert evaluations indicate that HuatuoGPT-II outperforms GPT-4 in 38% of test cases and ties in another 38%.

Me-Llama is a family of domain-specific LLMs designed for medical applications based on the Llama 2 architecture that includes foundation models Me-Llama 13B/70B and their chat-enhanced versions Me-Llama 13B/70B-chat [55]. The models were continually trained on 129B tokens sourced from biomedical literature, clinical notes, and general domain data. The data distribution aimed to balance catastrophic forgetting while enhancing medical knowledge retention. Then, the models were fine-tuned using 214K medical instruction samples with various tasks such as question answering, relation extraction, summarization, and named entity recognition and were tested on Medical In-context Benchmark Evaluation (MIBE), which spans six tasks and 12 datasets across biomedical, clinical, and general medical domains: question-answering, named entity recognition, relation extraction, classification, summarization, and natural language inference. Me-Llama outperformed ChatGPT on 7 out of 8 datasets and GPT-4 on 5 out of 8 datasets in task-specific instruction tuning. Additionally, it surpassed Llama 2 and other medical LLMs on 9 out of 12 datasets in classification, question-answering, and summarization tasks.

Additionally, in recent research works, instruction-tuned LLMs are adopted for domain-specific tasks via prompt engineering. DiabeTalk is a chatbot based on WizardLM to support diabetes diagnosis and education [56]. WizardLM is a family of general-purpose instruction-tuned LLMs built on top of Llama and was developed to enhance Llama's performance on complex instruction-following tasks using Evol-Instruct [57]. DiabeTalk was built without any further instruction tuning. Instead, it employs a prompt engineering strategy on WizardLM where medical questions related to diabetes diagnosis are structured into well-crafted prompt templates to get medically informed responses. The system integrates rule-based verification modules to validate the chatbot's outputs against known diabetes guidelines and risk indicators. Additionally, DiabeTalk supports few-shot prompting to be able to generalize across patient profiles using example-based in-context learning. The model was tested on a curated clinical dataset consisting of 200 synthetic patient profiles, each labeled by domain experts with ground-truth diabetes types. The predictions were compared against two baselines: a WE-LSTM classifier trained on the same data, and human clinician responses. DiabeTalk achieved an F1-score of 0.92, outperforming the WE-LSTM

with an F1 score of 0.87, and approximated human-level diagnostic consistency.

Despite targeting similar medical NLP tasks such as diagnostic text generation, clinical question-answering, and patient-doctor dialogue modeling, the fine-tuning strategies of these models vary considerably based on task complexity, language modality, and deployment constraints. Early models like BioBERT and SciFive use classic supervised or span corruption-based fine-tuning to improve performance on benchmark biomedical tasks, while newer models such as DoctorGLM and Me-Llama emphasize instruction tuning and parameter-efficient methods like LoRA to enable dialogue generation and real-time adaptability. Med42-v2 introduces Direct Preference Optimization for safer clinical outputs in contrast with Radiology-GPT's domain-specific LoRA tuning for structured report generation. ChatDoctor and HuatuoGPT-II further augment instruction tuning with retrieval or hybrid data-format unification to improve reliability and factual accuracy. These differences indicate the influence of the structure of the training data, the availability of resources, and the clinical usability in the real world on the choice of the fine-tuning strategy in medical LLM.

C. WRITING

The fine-tuned LLMs for writing application are listed in Table 3. CoPoet, a set of fine-tuned LLMs on T5 variants, including T5-3B, T5-11B, and T0-3B, was introduced as a collaborative poetry-writing system designed to generate poetic text based on user-provided instructions specifying content, rhyme, or rhetorical devices [58]. Unlike traditional auto-completion approaches, CoPoet is instruction-tuned on a specialized dataset of poetry-related instruction-output pairs. This dataset was created using rule-based templates and reverse-engineering instructions from publicly available poems and creative writing forums with a focus on training the model to respond to instructions such as adding metaphors, altering tone, or shifting style. The fine-tuning process employed cross-entropy loss optimization with hyperparameter tuning, and models were trained with a sequence length limit of 64 tokens using GPUs and techniques like DeepSpeed for computational efficiency. During inference, adjustments to top-k sampling, which restricts output selection to the k most probable tokens, and temperature, which controls randomness, were applied to enhance creativity and diversity in outputs. The performance of CoPoet was evaluated through both automated metrics and human assessments. Automated evaluations measure instruction-constraint satisfaction, while fluency and creativity are assessed through human judgments. CoPoet outperforms InstructGPT in satisfying poetic constraints, while InstructGPT remains more fluent overall. In compositional tasks, the T5-11B-poem variant satisfied 77.6% of instructions compared to InstructGPT's 55.2%. Furthermore, a user study involving 15 expert crowd workers also showed that CoPoet-assisted poems were preferred over solo-written

poems in 57% of cases, with users frequently incorporating CoPoet's outputs into their final work.

CoEdit [60] is a domain-specific language model based on FLAN-T5 for instruction-based text editing and writing assistance in three different sizes: CoEdit-L with 770M parameters, CoEdit-XL with 3B parameters, and CoEdit-XXL with 11B parameters. The model is trained on an 82,000-pair dataset derived from ITERATER+, a framework that helps the model understand and predict the types of changes that users might make to a document. During training, CoEdit was given an initial version of the text along with various edits applied by users. The model learns to predict an edited version of the text conditioned on a natural language instruction. Then, the model was fine-tuned to handle user instructions, where users provide high-level guidance on how the text should be changed. For example, instructions like "make this more formal" or "simplify this section" were used to train the model on how to make targeted edits. The model was evaluated using standard text editing benchmarks, including grammatical error correction datasets (JFLEG), text simplification datasets (ASSET and TurkCorpus), coherence evaluation sets (DISCOFUSE), and style transfer corpora (GYAFC and WNC). CoEdit consistently outperformed baseline instruction-tuned LLMs such as PEER, InstructGPT, and GPT-3 Edit in automated metrics and human evaluations. Human evaluation studies indicated that CoEdit-generated edits were preferred by users 64% of the time over GPT-3 Edit, demonstrating its effectiveness in practical writing assistance. However, challenges remain, such as prompt sensitivity, where different instruction phrasings can affect the model's performance. Additionally, while CoEdit excels in sentence-level text editing, its applicability to long-form document revision is yet to be explored.

SkipWriter is a fine-tuned LLM-powered system that supports abbreviated handwritten input on tablets by converting variable-length prefix-based handwritten abbreviations into complete digital phrases. [62]. For example, if the user writes the abbreviation "t y f y s" (short for "thank you for your support"), SkipWriter decodes and expands it to the intended full phrase. The system integrates an in-production fine-tuned variation of PaLM2 to decode abbreviations into full phrases. The model was fine-tuned using a dataset in which full phrases were transformed into variable-length prefix-based abbreviations using an entropy-based abbreviation strategy that simulates human shorthand behavior. Fine-tuning was conducted on 64 TPU v4 chips with AdaFactor optimizer and using a SentencePiece tokenizer to minimize the cross-entropy loss on the full-phrase outputs. The dataset included diverse sources, such as scientific papers, Wikipedia articles, news content, and movie reviews. The fine-tuned LLM was trained with a global batch size of 256 and a dropout set to 0.2. This configuration allowed the model to learn efficiently, with a maximum input length of 512 tokens and an output length of 128 tokens. The system effectively reduced motor movements during handwriting by up to

TABLE 3. Fine-tuned models for writing applications.

Fine-tuned Model	Base Model	Dataset	Dataset Size	Model Size	Computational Resources	Evaluation Metric/Framework
CoPoet [58]	T5 variants [33]	Synthetic instruction–poem pairs generated from poetry corpora and creative writing sources. [59]	873,574	3B/11B	Multiple A100 GPUs	Instruction-constraint satisfaction, and human evaluation of creativity, fluency, and preference
COEDIT [60]	FLAN-T5 variants [34]	Instruction–edit pairs derived from multiple text-editing benchmarks using the ITERATER+ taxonomy [61]	82K	770M/3B/11B	A cluster of A100 GPUs using Deep-speed	GLEU, SARI, Accuracy, and human evaluation
SkipWriter [62]	PaLM2 (in-production variant) [63]	Scientific Papers from ArXiv and PubMed Open Access, Wikipedia articles, Multi-News articles from Newser, and IMDB reviews	Not specified	Not specified	64 TPU v4 chips	Words per minute (WPM), word error rate (WER), traversal distance per character (TDC), and user study evaluations

60%, measured using traversal distance per character, while maintaining a writing speed of 25.78 words per minute. Furthermore, an offline simulation estimated the upper bound of abbreviation expansion efficiency and revealed that users effectively utilized 90% of the model’s abbreviation expansion potential. However, challenges remain in optimizing abbreviation strategies, ensuring high decoding accuracy across different handwriting styles, and balancing abbreviation efficiency with cognitive load. Further refinements, including on-device model compression and personalized adaptation, could enhance SkipWriter’s usability in real-world applications.

In the writing domain, fine-tuning strategies are aligned with the need for creativity, user interactivity, and context-sensitive editing. CoPoet emphasizes instruction tuning on poet-specific prompts and creative constraints, targets artistic text generation, and leverages sampling strategies like top-k and temperature control to foster diversity. In contrast, CoEdit extends fine-tuning to instruction-based text editing tasks by learning revision patterns from instruction–edit pairs, thus combining structural prediction with instruction-following, making it ideal for iterative refinement and writing assistance. SkipWriter diverges further by incorporating prefix-based abbreviation decoding for handwritten input, using supervised fine-tuning on a synthetic abbreviation expansion dataset tailored for accessibility and speed. While CoPoet is optimized for aesthetic fluency and style adaptation, CoEdit focuses on sentence-level precision and user-aligned edits, and SkipWriter prioritizes practical usability in real-time writing scenarios. These models demonstrate how instruction tuning, fine-grained task formulation, and in some cases, multimodal adaptation are customized to address the

varied goals of writing assistance, from artistic generation to efficiency-enhancing abbreviation decoding.

D. EDUCATION

Education has also been a key area of interest for fine-tuning LLMs, as shown in Table 4. EdBERT is a fine-tuned version of BERT for educational applications, specifically for handling sentiment analysis tasks within virtual learning environments [64]. The model was designed to assess student feedback and interactions by classifying text according to emotional tone, which is crucial for understanding student engagement, satisfaction, and emotional well-being in online learning settings. This can be used to adjust teaching strategies, provide additional support to struggling students, or enhance learning resources based on student feedback. The model pretrained on an educational dataset consisting of student reviews from online learning platforms. The fine-tuning process involved preparing a dataset of student interactions, where the text was preprocessed to remove irrelevant symbols and noise and then tokenized to be compatible with BERT-based models. This dataset was labeled with sentiment categories such as positive, negative, and neutral. Then, the model was trained to learn the relationships between student feedback and these sentiment labels to accurately classify the emotional tone of new, unseen text. This process involved supervised learning with a cross-entropy loss function, AdamW optimizer, and a softmax activation function to classify sentiments. The model was evaluated with accuracy (87.89%), precision (89%), recall (88%), and F1-score (88%) and outperformed existing sentiment classification models. A key strength of EdBERT is its integration with ensemble learning methods, such as

TABLE 4. Fine-tuned models for education applications.

Fine-tuned Model	Base Model	Dataset	Dataset Size	Model Size	Computational Resources	Evaluation Metric/Framework
EdBERT [64]	BERT-Large [31]	Student interactions on Coursera	4,823	340M	Mac-M1 CPU	Accuracy, F1 Score, Precision, Recall
SciEdBERT [65]	BERT [31]	Academic Journal Dataset for Science Education (SciEdJ).	2K journal articles	110M	Not specified	Accuracy
		Students' written Responses to 49 assessment tasks (SR1) where seven tasks used for fine-tuning	~ 50K			
		Students' responses to argumentation tasks (SR2) where four tasks used for fine-tuning	2,940			
G-SciEdBERT [66]	G-BERT	PISA 2015 [67] & PISA 2018 [68] re-sults	30K for pretaining and 20K for fine-tuning	110M	NVIDIA RTX 8000 GPU	Quadratic Weighted Kappa (QWK)
GPT-3.5-Turbo for automatic scoring [69]	GPT-3.5 Turbo [3]	Expert-scored NGSS-aligned science assessment responses from middle- and high-school students	8,690	Not Specified	OpenAI's API in-frastructure	Accuracy, Mean Absolute Error, Paired-sample t-tests
SciDeBERTa [70]	DeBERTa [71]	SciERC [72]	2,867 sentences	86M	8 A100 (80GB) GPUs for pretraining and 2 RTX 2080 GPUs for fine-tuning	F1 Score
SciDeBERTa(CS) [70]		Genia [73]	18,545 sentences			

a random forest classifier, which further refines predictions. However, similar to previous LLMs that are built through classic supervised fine-tuning, EdBERT lacks conversational abilities in modern LLMs.

SciEdBERT is a family of BERT-based language models designed for science education tasks such as the automated scoring aligned with expert-assigned labels of student responses and the content analysis of scientific assessments [65]. It also assists in content analysis by processing scientific texts for classification and concept identification. This allows it to handle a variety of tasks, such as recognizing key scientific concepts and categorizing content for educational purposes. The model is built through continual pretraining on science education corpora, including 2,000 journal articles from top science education journals, 50,000 student-written science responses, and 2,940 student responses to argumentation tasks. Then, the model is fine-tuned on automated scoring and concept classification tasks using datasets of student responses labeled by trained experts. This process follows a BERT-base architecture

with 12 layers and a WordPiece tokenizer, optimized using AdamW, with cross-entropy loss applied for classification tasks. SciEdBERT was evaluated across multiple automated scoring benchmarks and was compared with baseline models like BERT, SciBERT, and domain-specific variants trained on science education data. The best-performing SciEdBERT variant achieved an average accuracy of 0.866 in science assessment tasks. Challenges remain in handling long-form, multi-step scientific explanations, where context dependency and reasoning complexity can limit the model's accuracy. Additionally, the model is optimized for English-language science education, and its applicability to multilingual education remains an open research question.

Followed by this trend, G-SciEdBERT is similarly built upon G-BERT, a German-language BERT model, for automated scoring of German-written student responses in science education [66]. Similar to the SciEdBERT training process, this model was continually pretrained on 30,000 German student responses from the PISA 2018 dataset. Then, the model was fine-tuned on an additional 20,000 responses

(2M tokens) from PISA 2015 using supervised learning with a categorical cross-entropy loss. G-SciEdBERT uses structured input formats that incorporate task descriptions, answer templates, and contextual examples to distinguish between various levels of student understanding. Evaluation is conducted using quadratic weighted Kappa (QWK), where G-SciEdBERT achieved a 10.2% improvement over G-BERT.

GPT-3.5 was also developed for the automatic scoring of student responses in science education with a focus on multilabel and multiclass assessment tasks [69]. The model was fine-tuned using datasets collected from middle and high school students who participated in science assessments where responses were manually scored by experts to provide ground truth labels. The fine-tuning process involved using labeled student responses as inputs, where each response is associated with expert-defined rubric scores. The tasks were designed to assess the students' scientific knowledge and reasoning skills, such as explaining physical phenomena or performing calculations related to energy and matter. In the fine-tuning process, the GPT-3.5-Turbo model was initialized with its pretrained weights and then exposed to these domain-specific datasets. The fine-tuned GPT-3.5-Turbo model was compared against a fine-tuned BERT baseline and demonstrated an average accuracy improvement of 9.1% across all tasks. In multi-label tasks, GPT-3.5-Turbo achieved a 7.1% improvement over BERT in more complex assessment items, while it outperformed BERT by 10.6% on average in multi-class tasks. A paired-sample t-test confirmed that the fine-tuned GPT-3.5's accuracy gains were statistically significant with $p < 0.05$. The study also emphasizes the need for further research into ethical considerations and broader applications of fine-tuned LLMs in education.

SciDeBERTa and SciDeBERTa(CS) are also introduced as domain-specific language models fine-tuned based on DeBERTa and specialized in tasks involving both scientific and technological terminology and knowledge extraction, such as named entity recognition, relation extraction, and co-reference resolution [70]. After pretraining SciDeBERTa and SciDeBERTa(CS) on a general corpus and then specifically in the science and technology domains, the models were fine-tuned. The fine-tuning process involved reinitializing specific layers of the DeBERTa model to prevent overfitting. The top layers of the model were re-initialized during fine-tuning, which allowed the model to adapt better to the domain-specific tasks of extracting information from scientific texts. In addition to layer re-initialization, various optimizers, such as AdamW and RAdam, were compared to determine the best optimization strategy for improving performance. They found that RAdam provided the most stable and effective training performance without the use of a learning rate scheduler. SciDeBERTa was fine-tuned on two datasets, SciERC and Genia. SciERC consists of abstracts from AI conference papers, whereas Genia contains abstracts from Medline. These datasets enabled the model to be trained for tasks specific to scientific domains, where it achieved

state-of-the-art or outperformed models, such as SciBERT and S2ORC-SciBERT, in most tasks.

Fine-tuning strategies in educational LLMs vary depending on task specificity, language coverage, and the balance between classification and generative outputs. Early models like EdBERT and SciEdBERT adopt classic supervised fine-tuning for sentiment classification and automated scoring as they benefit from BERT's strong contextual embeddings while integrating ensemble or regression techniques to boost task performance. In contrast, multilingual models, such as SciEdBERT, and generative models, such as GPT-3.5-Turbo, leverage continual pretraining and careful low-rate fine-tuning to enhance language-specific domain adaptation and multilabel classification, respectively. GPT-3.5, in particular, introduces few-shot generative capabilities into scoring tasks and marks a shift from static classifiers to flexible, instruction-following architectures. Meanwhile, SciDeBERTa models explore structural fine-tuning techniques such as layer re-initialization and optimizer tuning to extract scientific entities and relations more effectively. These strategic variations reveal how the nature of educational data—whether sentiment-labeled, rubric-scored, or concept-dense, drives the design of fine-tuning pipelines.

E. AGRICULTURE

Fine-tuned LLMs for agricultural applications are listed in Table 5. AgriBERT is a domain-specific language model designed for applications in the agricultural and food sectors [74]. This model is built on the BERT architecture and was created to address the challenge of linking food descriptions from retail scanner data to nutrition information from databases, such as the USDA's Food and Nutrient Database for Dietary Studies. This link is important for studying the relationship between retail food purchases and community health. AgriBERT was trained from scratch using a domain-specific corpus consisting of more than 46,000 journal papers related to agriculture and food with over 300 million tokens. Then, the model was fine-tuned for a downstream task formulated as an answer selection problem using the MLM method to treat the task of mapping food descriptions to nutritional information as a question-answering problem. The task involved mapping product descriptions from retail scanner data (e.g., Nielsen) to the corresponding nutritional descriptions from the USDA database. The model is trained to select the correct USDA description (answer) for each retail product description (question). Additionally, AgriBERT incorporates external knowledge from the FoodOn ontology, a structured database of food-related terms. This knowledge-infusion technique helps the model make more accurate associations between food descriptions and nutrition labels. AgriBERT was evaluated on the answer selection task, where it demonstrated superior performance compared to baseline models such as general-purpose BERT and traditional k-nearest neighbor methods. The model achieved a Mean Average Precision score of 44.2%. However, the incorporation of external

TABLE 5. Fine-tuned models for agriculture applications.

Fine-tuned Model	Base Model	Dataset	Dataset Size	Model Size	Computational Resources	Evaluation Metric/Framework
AgriBERT [74]	BERT [31]	USDA Food Acquisition and Purchase Survey	32K	110M	Not specified	Precision@1, Mean Average Precision
ShizishanGPT [75]	GPT-4 [30]	Question-answer pairs related to agricultural topics	100	>175B	Not specified	Accuracy, Professionalism, Language Fluency, BLEU, ROUGE, GLEU, Composite Score
IPM-AgriGPT [76]	ChatGLM3 [77]	Web sources, databases, and textual documents related to pest and disease management	5,704	6B	NVIDIA A800 80GB GPU, 14-core Intel Xeon Gold 6348, 100GB RAM	Precision, Domain-Specific Professionalism, Safety, and Effectiveness

knowledge had mixed results: while precision-at-1 (P@1) improved to nearly 50% with the addition of FoodOn entities, the overall MAP score slightly decreased. This suggests that knowledge infusion can enhance specific aspects of model performance but may also introduce ranking inconsistencies in semantic matching tasks.

ShizishanGPT, which is based on the GPT model, is another LLM for agricultural applications [75]. ShizishanGPT integrates a RAG framework, an agricultural knowledge graph (AgriKG), and external tools for tasks like phenotype prediction. The model is fine-tuned on a dataset of 100 high-quality agricultural question answering pairs and retrieves domain-specific knowledge to generate accurate and professional responses. This dataset covered diverse topics, including crop cultivation, pest control, agricultural technology, and processing and was collected from existing agricultural knowledge bases, academic papers, and websites. The model architecture follows a ReAct (Reasoning and Action) framework that enables dynamic problem-solving by using feedback to adjust the course of action when handling complex queries. For example, when a user asks about maize growth rates based on certain DNA sequences, Shizishan GPT uses prompts designed specifically for agricultural data, accessing knowledge graphs, and prediction models to deliver accurate answers. ShizishanGPT achieved a high composite score of 0.923 and outperformed models like GPT-4 and Claude in BLEU, ROUGE, and GLEU metrics. Expert reviews rated its accuracy as 9.8/10 and professionalism as 9.5/10.

Another LLM in agriculture is IPM-AgriGPT designed for pest and disease management [76]. This Chinese LLM incorporates a Generation-Evaluation Adversarial framework to generate high-quality question-answer corpora. Additionally, it employs Agricultural Contextual Reasoning Chain-of-Thought Distillation, which transfers reasoning steps from a teacher model to a student model and improves decision-making in complex scenarios. IPM-AgriGPT was

built on ChatGLM3-6B as a base model due to its balance between parameter efficiency and performance in Chinese-language tasks. The model was trained using a specialized dataset of 5704 entries collected from web sources, databases, and textual documents related to pest and disease management. During training, the model used supervised fine-tuning with a cross-entropy loss function. The model was evaluated using a domain-specific benchmark that assesses professionalism, safety, and effectiveness in pest control recommendations. Comparative analysis with GPT-3.5-Turbo, Baichuan2-7B, and ChatGLM3-6B demonstrated that IPM-AgriGPT performs competitively, particularly in its ability to generate detailed, context-aware responses for pest and disease-related queries.

Fine-tuning strategies for agricultural LLMs are shaped by the domain's need for structured knowledge integration, precision in matching heterogeneous data, and contextual reasoning in specialized queries. AgriBERT adopts a classic BERT-style pretraining on a large domain-specific corpus and frames downstream tasks like food description matching as answer selection problems, further enhanced by ontology-based knowledge infusion using FoodOn. In contrast, ShizishanGPT leverages a RAG architecture and ReAct prompting to dynamically integrate knowledge graphs and tools during response generation, which offers high accuracy in complex agricultural question answering tasks. IPM-AgriGPT further advances reasoning capabilities by incorporating Chain-of-Thought distillation from teacher models and a Generation-Evaluation Adversarial framework to bootstrap high-quality question answering datasets. These diverse approaches reflect the emphasis of the agricultural sector on integrating scientific knowledge, domain safety, and adapting to multilingual, real-world advisory needs.

F. LEGAL

Researchers have worked on fine-tuning LLMs for legal purposes, as shown in Table 6. LegalBERT is a family of

TABLE 6. Fine-tuned models for legal applications.

Fine-tuned Model	Base Model	Dataset	Dataset Size	Model Size	Computational Resources	Evaluation Metric/Framework
LEGAL-BERT [78]	BERT-base [31]	EURLEX57K (legislative documents) [79]	57K	110M	v3 TPUs with 8 cores from Google Cloud Compute Services	Perplexity-based F1 Score on end-task results
		ECHR-CASES [79]	11.5K			
		CONTRACTS-NER (US contracts from EDGAR) [79]	2K			
ChatLaw [80]	Not Specified, but instruction data for a MoE-based legal LLM is formatted using Llama chat template.	Legal samples of case classification, statute prediction, and legal document drafting	4M	Not specified	8 80GB A100 GPUs and two Intel Xeon 8358P processors	LawBench, Unified Qualification Exam for Legal Professionals
LawyerLlama [81]	Chinese Llama	Chinese expert-labeled legal question answering	850	Not specified	8 80GB A100 GPUs and two Intel Xeon 8358P processors	Perplexity-based zero-shot accuracy, Charge Prediction, National Judicial Examination (marriage subset), human evaluation
LawGPT [82]	Chinese-Alpaca-Plus 7B model (Llama-7B-based)	Combined 200,000 samples from an open-source crime case dataset, 20,000 samples from the JEC-QA dataset, and 80,000 ChatGPT-refined legal instruction samples	300K (30K used for fine-tuning)	7B	8 NVIDIA V100 GPUs	Accuracy-based task scores
DISC-LawLLM [83]	Baichuan-13B-Base	DISC-Law-SFT [83]	403K	13.2B	8 A800 80GB GPUs	DISC-Law-Eval, Accuracy (objective), Completeness, Clarity (subjective)
LawLLM [84]	Gemma-7B [85]	CaseLaw [86]	1M for training and 200K for testing	7B	NVIDIA GPUs A40	Top-k accuracy (Top-1/3/5), Not-Found rate (hallucination), Accuracy, F1

domain-specific language models based on the BERT model for legal-text processing [78]. multiple domain-adaptation strategies have been investigated using BERT, including using BERT out of the box, further pre-training BERT on large-scale legal corpora, and pre-training BERT from scratch with a legal-specific vocabulary. In the domain-adaptation setting, BERT is trained using Masked Language Modeling and Next Sentence Prediction on legal texts such as court decisions, contracts, and legislation from multiple public legal corpora, including datasets from the European Court of Human Rights (ECHR) and U.S. court cases. Following domain adaptation, LegalBERT

models are fine-tuned separately across several legal NLP tasks, including multi-label text classification (EURLEX57K dataset), binary and multi-label classification of legal cases (ECHR-CASES dataset), and named entity recognition on legal contracts (CONTRACTS-NER dataset). LegalBERT has multiple variants, including LEGAL-BERT-SC with 100 million parameters, which uses the BERT-BASE architecture with 12 layers, 768 hidden units, and 12 attention heads. Additionally, a more compact version, LEGAL-BERT-SMALL with 35 million parameters, was created with only 6 layers, 512 hidden units, and 8 attention heads. Despite its smaller size, LEGAL-BERT-SMALL trains approximately

four times faster and requires fewer hardware resources while still delivering competitive performance in legal tasks. Evaluation results showed that LegalBERT outperforms general BERT models across multiple legal NLP tasks, particularly in multi-label classification and contract element extraction. However, handling ambiguous legal terminology, cross-jurisdictional legal variations, and reasoning transparency in legal decision-making remains challenging for this model.

ChatLaw is another legal LLM to assist users in legal consultations by integrating a Mixture-of-Experts (MoE) framework at the model-level and a multi-agent system at inference time [80]. In an MoE model, the neural network is divided into multiple specialized subnetworks or experts, and only a small subset of them is activated during each inference step. The multi-agent system operates during inference where multiple role-specific LLMs interact in a structured workflow, with each agent specializing in part of the system's tasks. ChatLaw was fine-tuned using an instruction-based approach within the MoE framework on a diverse legal dataset with approximately 4 million samples across 10 major and 44 minor categories. This dataset was formatted for MoE models to help the model perform case classification, statute prediction, document drafting, and more specialized tasks, such as public opinion analysis and named entity recognition. The Chatlaw framework consists of four specialized roles: Legal Assistant, Legal Researcher, Lawyer, and Legal Editor, each responsible for distinct aspects of legal consultation. The Legal Assistant gathers user information and builds a legal knowledge graph, the Legal Researcher retrieves statutes and relevant case law using RAG, the Lawyer provides structured legal advice, and the Legal Editor refines responses and drafts legal documents. This model was evaluated on benchmarks like LawBench and the Unified Qualification Exam for Legal Professionals and achieved the score of 60.08 on LawBench (compared to GPT-4's 52.35) and 115 on the Qualification Exam (GPT-4 scored 104). The model was also tested in real-world legal consultation settings, where it received the highest expert ratings for completeness, correctness, legal guidance, and authority compared to competing models.

LawyerLlama is a Chinese legal LLM designed for legal reasoning and consultation [81]. LawyerLlama follows a three-step adaptation process: continual pretraining on a Chinese legal corpus, supervised fine-tuning on expert-written legal solutions, and integration of a retrieval module that dynamically incorporates relevant legal texts before answering queries. The pretraining phase involved further training Llama on legal documents, court rulings, and judicial interpretations collected from Chinese legal sources. This was followed by supervised fine-tuning on a dataset of expert-labeled legal questions and solutions, and the retrieval module fetches statutes, regulations, and precedents from an external database before generating responses. LawyerLlama was evaluated using zero-shot accuracy on legal benchmarks such as Charge Prediction and the National Judicial Examination (marriage subset), as well as human evaluation on real-world legal consultation questions. However, its scope

remains limited, as it primarily focuses on marriage-related legal cases within the Chinese judicial system, which makes it less generalizable to other legal subfields.

LawGPT is an open-source Chinese LLM for legal reasoning, judgment prediction, document retrieval, and legal question answering [82]. The model addresses two key limitations in applying general LLMs to legal tasks: the lack of domain-specific legal knowledge in open-source models, which hinders their performance in legal applications, and the privacy concerns associated with proprietary models like GPT-4 and GPT-3.5, which may not be suitable for sensitive legal consultations. To mitigate these issues, LawGPT is built on the Chinese-Alpaca-Plus 7B model (Llama-7B-based) and integrates legal-oriented pretraining and supervised fine-tuning using large-scale Chinese legal documents. The pretraining phase of LawGPT involved 500,000 legal documents that cover civil law, criminal law, and administrative law. Then, legal-supervised fine-tuning was applied, where the model was trained on a subset of 300,000-instance knowledge-driven instruction dataset. This dataset was divided into three subsets including an open-source legal dataset with 200,000 crime-related cases with crime type prediction and legal consultation tasks, the JEC-QA dataset with 20,000 samples for legal question answering, and a curated legal dataset with 80,000 cases refined using ChatGPT. LoRA technique was also used with different configurations across stages. LawGPT is evaluated in a zero-shot setting on eight legal tasks, including article prediction, charge prediction, prison term prediction, case analysis, and consultation, and compared against GPT-3.5, GPT-4, and Llama-7B. While GPT-4 consistently outperformed all models, LawGPT outperformed Llama 7B on major legal tasks. However, despite these improvements, LawGPT still lags behind some LLMs in complex legal reasoning and advanced judgment prediction tasks.

DISC-LawLLM is another fine-tuned Chinese legal LLM based on the Baichuan-13B-Base, a 13.2-billion-parameter model pretrained on a 1.4-trillion-token corpus [83]. The model was built through supervised fine-tuning with a dataset called DISC-Law-SFT, which contains over 400,000 samples focused on legal tasks such as legal element extraction, judgment prediction, judicial examination, document summarization, and question-answering. The fine-tuning process involved a global batch size of 256 and a learning rate of 5×10^{-5} running over two epochs on eight A800 GPUs. The input sequence length was set to a maximum of 2048 tokens, with output sequences limited to 1024 tokens to handle extensive legal documents. To further improve the accuracy of real-world applications and access to external legal knowledge, the system can be augmented with a retrieval module. This module encodes legal statutes, court rulings, and other regulatory texts as vectors and retrieves the most relevant ones when responding to queries. Additionally, to enhance legal reasoning, the DISC-LawLLM leverages law-specific reasoning inspired by legal syllogisms. The effectiveness of DISC-LawLLM was evaluated using a newly proposed

benchmark, DISC-Law-Eval, which includes multiple-choice legal examinations of varying difficulty levels, as well as subjective assessments based on human evaluations. The model outperformed other domain-specific legal LLMs, including ChatLaw, LawyerLlama, and LawGPT, as well as general-purpose models like GPT-3.5-Turbo, on the majority of objective and subjective evaluation tasks.

LawLLM is an LLM for legal tasks in the U.S. legal system, particularly for Similar Case Retrieval, Precedent Case Recommendation, and Legal Judgment Prediction [84]. To build this model, Gemma-7B was fine-tuned using instruction tuning on U.S. legal cases from the CaseLaw (Case Access Project) dataset with task-specific preprocessing to focus on accurately distinguishing and interpreting legal nuances, precedent relationships, and judgment patterns. While traditional retrieval models rely primarily on textual similarity, LawLLM incorporates legal reasoning mechanisms, distinguishing between precedent cases that have been considered in judicial decisions and merely similar cases with overlapping textual content, which helps the model offer more reliable case recommendations and predictions that align with legal principles rather than just statistical similarity. The fine-tuning process involved instruction tuning using a 4-bit quantized LoRA approach with a cross-entropy loss, over ten epochs with a maximum input length of 4096 tokens. To fine-tune the model, first, raw legal cases are structured into machine-readable formats. Then, vector databases and knowledge graphs are used to enhance retrieval accuracy, and LawLLM learns structured tasks such as predicting legal outcomes and retrieving precedent cases through instruction tuning. LawLLM was evaluated against GPT-3.5, GPT-4, Llama 2 7B, Vicuna 13B, and Guanaco-13B on three core legal tasks using top-k accuracy, hallucination rate, and accuracy/F1 metrics. Results demonstrated that LawLLM outperformed all baselines with higher retrieval accuracy (top-1, top-3, and top-5 metrics) and lower hallucination rates in Similar Case Retrieval and Precedent Case Recommendation. In Legal Judgment Prediction, LawLLM surpassed GPT-4 with a 6.3% improvement in accuracy in zero-shot settings and a 6.2% increase in few-shot learning scenarios.

The diversity of legal language, jurisdictional variance, and the high stakes of legal reasoning have led to a wide spectrum of fine-tuning strategies for legal LLMs. Foundational models like LegalBERT rely on classic BERT-based pretraining and supervised fine-tuning using legal corpora for classification and named entity recognition tasks to maintain a balance between interpretability and performance. In contrast, newer systems like ChatLaw and DISC-LawLLM employ instruction tuning, Mixture-of-Experts architectures, and retrieval augmentation to handle multi-role consultation, long-document reasoning, and statute integration. Chinese models such as LawyerLlama and LawGPT emphasize retrieval-enhanced generation and LoRA-based adaptation to optimize for resource efficiency and privacy in localized legal systems. LawLLM integrates legal reasoning into retrieval

tasks by instruction tuning on structured U.S. case law, pushing beyond lexical similarity toward jurisprudential relevance. These methodological choices reflect legal LLMs' need to handle formal logic, domain-specific reasoning, and explainable decision-making, often trading off between model complexity, deployment feasibility, and jurisdictional focus.

G. SOFTWARE DEVELOPMENT

LLMs have become a source of help in programming and software development as well. Table 7 lists fine-tuned LLMs for this domain.

The BigCode community has introduced StarCoder and StarCoderBase [87], two LLMs with 15.5 billion parameters and an 8K context length fine-tuned for code generation and code completion. StarCoderBase is trained on one trillion tokens from The Stack, which is a large collection of permissively licensed GitHub repositories. StarCoder is fine-tuned on 35 billion Python tokens, which enables it to perform especially well on Python-related tasks while still retaining its ability to handle multiple programming languages. The fine-tuning process was based on a Fill-in-the-Middle approach, where StarCoder was trained not only to predict the next token but also to generate code that completes sections in the middle of the code blocks. Furthermore, Multi-Query Attention, as a variant of the attention mechanism where all query vectors share a single set of key and value vectors, was used to enable large-batch inference for long sequences with up to 8K context lengths. The model underwent extensive fine-tuning in 8,500 training steps using the Adam optimization algorithm with state-of-the-art evaluation techniques and benchmarks, including zero-shot and few-shot code generation and infilling tasks. The evaluation showed that StarCoderBase outperformed existing open LLMs for code, including OpenAI's code-cushman-001 model. The models are released under the OpenRAIL-M license to address safety concerns, including improved PII reduction and an attribution tool.

WizardCoder is a fine-tuned version of StarCoder 15B and CodeLlama-34B-Python that incorporates instruction fine-tuning through the Evol-Instruct method adapted for the programming domain [88]. This method handles more complex and diverse code instructions by addressing code debugging and time-space complexity considerations. The training dataset was iteratively generated using Evol-Instruct on the Code Alpaca dataset, initially with 20,000 instruction-following samples, which were iteratively evolved into 78,000 samples of instruction, input (code snippet), and expected output attributes. The evol-instruction prompts are adapted for the code domain which streamlines instructions, simplifies templates, and incorporates code-specific elements. The fine-tuning process used a cosine learning rate scheduler, a batch size of 512, a learning rate of 2×10^{-5} , and mixed precision. WizardCoder-15B surpasses closed-source models such as Claude and Bard on HumanEval and HumanEval+, while WizardCoder-34B

TABLE 7. Fine-tuned models for programming applications.

Fine-tuned Model	Base Model	Dataset	Dataset Size	Model Size	Computational Resources	Evaluation Metric/Framework
StarCoder [87]	StarCoderBase	GitHub Repositories	12K	15.5B	64 NVIDIA A100 GPUs	HumanEval, MBPP, and DS-1000 benchmarks
WizardCoder [88]	StarCoder-15B [87] and CodeLlama-34B-Python [89]	Code Alpaca	78K samples	15B/34B	Not specified	HumanEval, HumanEval+, MBPP, DS-1000, MultiPL-Es
OctoPack [90]	StarCoder [87]	Code Alpaca	78K samples	15B	Not specified	HumanEval, HumanEval+, MBPP, and DS-100 benchmarks
Llama-Reviewer [91]	Llama [4]	Tufano et al. Java Code Review [93]	~ 168K	6.7B	NVIDIA A100-SXM4 80GB GPUs	Precision, Recall, F1 Score, BLEU-4
		CRer multi-language code review [93]	~ 600K			
CodeT5+ (fine-tuned for evaluation) [94]	T5 and CodeGen-mono [95] [33]	Multilingual GitHub code and CodeSearchNet (bimodal)	51.5B tokens	220M/770M/2B/6B/16B	16 NVIDIA A100 40G GPUs for pre-training	BLEU-4, CodeBLEU, EM, Accuracy, F1 (task-dependent)
Fine-tuned T5-small [96]	T5-small [33]	Tufano et al. [92], Watson et al. Test-Assert Pairs [97], and Haque et al. Java methods with summaries [98]	~ 2.67M	60M	8 TPU cores and 35.5GB of RAM	Accuracy@K, BLEU-A, ROUGE-LCS
Lemur-70B-Chat [99]	Llama 2 [4]	The Stack, RefinedWeb, Wikipedia, Books, ArXiv, StackExchange, DM Mathematics	90B tokens	70B	TPUv4-512 pods	Accuracy, Exact Match, pass@1, Success Rate (agent benchmarks)
WaveCoder [99]	StarCoder-15B, CodeLLaMA-7B/13B, DeepSeekCoder-6.7B [36]	CodeSeaXDataset (code summarization, generation, translation, repair)	~ 20K	StarCoder: 15B, CodeLLaMA: 7B, 13B), DeepseekCoder: 6.7B	NVIDIA A100-80GB GPUs; tensor parallelism (StarCoder/CodeLLaMA); FSDP (DeepSeekCoder)	HumanEval, MBPP, HumanEvalFix, HumanEval-Explain, HumanEvalPack (pass@1)

achieves performance comparable to GPT-3.5 and surpasses it on HumanEval+.

OctoPack is another fine-tuned version of StarCoder for code generation, repair, and explanation [90]. The model is fine-tuned using COMMITPACK, a 4TB dataset of Git commits across 350 programming languages, from which a filtered 2GB high-quality instruction dataset (COMMITPACKFT) is derived. The instruction tuning process is conducted on COMMITPACKFT, alongside OASST (a multi-turn chat dataset) to improve the model's ability to handle diverse programming tasks. To evaluate model performance, HUMANEVALPACK is introduced which extends HumanEval to six languages (Python, JavaScript, Java, Go, C++, Rust) and covers three tasks: code synthesis, repair, and explanation. In this process, natural Git commit structures where commit messages serve as task

instructions and the corresponding code changes provide the expected outputs. A subset of 5,000 instruction samples is selected from COMMITPACKFT across the six evaluated languages. Evaluation is performed using pass@1, measuring correctness across multiple attempts. OCTOCODER, the fine-tuned StarCoder variant, achieves 46.2% pass@1 on Python HumanEval, outperforming all open-source models while remaining below GPT-4's state-of-the-art performance. This work demonstrates the effectiveness of using real-world code commits for instruction tuning as an alternative to synthetic instruction generation from proprietary models like OpenAI's.

Llama-Reviewer is a Llama-based framework that automates code review tasks and addresses resource constraints through parameter-efficient fine-tuning (PEFT) [91]. Llama-Reviewer first analyzes a code snippet to predict whether

a review is necessary. If so, it generates a constructive review comment. Based on this feedback, it then suggests refinements to improve the code. The model undergoes a two-stage fine-tuning process. First, it is instruction-tuned using a dataset focused on code-related tasks. Then, a supervised fine-tuning stage is applied to three core tasks: predicting whether a code change needs a review, generating code review comments, and refining code based on reviewer feedback. For this, Llama-Reviewer adopts Stanford Alpaca's instruction-following tuning methodology but replaces its natural language dataset with Code Alpaca for a better alignment with code review requirements. This process also involves using two PEFT techniques: Zero-Init Attention Prefix-Tuning and LoRA. Prefix-tuning adds special tokens to control multi-head attention in Llama's transformer layers without altering the base model's weights, and LoRA introduces trainable low-rank matrices that approximate full-parameter tuning while keeping the computational cost minimal. These techniques allow the model to retain high performance without the need for expensive full-model retraining. Llama-Reviewer was evaluated on two publicly available datasets: the CRer dataset, which contains multi-language and line-based code review data, and the Tufano dataset, which is focused on Java and function-level reviews. LLaMA-Reviewer achieved an F1 score of up to 70.49 in review necessity prediction and the highest BLEU-4 scores in review comment generation when using LoRA. In the code refinement task, it delivered results comparable to state-of-the-art models while requiring significantly fewer parameters. This framework demonstrates that unified LLMs, when combined with PEFT, can effectively replace task-specific pretrained models for code review automation under limited computational resources.

CodeT5+ is a family of open-source large language models for code understanding and generation, which is based on the T5 architecture and Codegen-mono [94]. CodeT5+ models were pretrained using large-scale unimodal code on multilingual GitHub data, followed by bimodal text-code pretraining on function-level datasets such as CodeSearchNet. CodeT5+ models were then evaluated in different settings, including fine-tuning, which was designed to address various code-related tasks such as code summarization, code generation, code translation, and code defect detection. This involved providing the model with training examples from these domains and adjusting the weights accordingly to specialize in code comprehension tasks. Fine-tuning for code translation involves translating code between different programming languages, while code summarization focuses on generating natural language descriptions for code snippets. Experimental results show that CodeT5+ models consistently outperform prior models such as GPT-2 and RoBERTa on benchmarks including CodeXGLUE and CodeSearchNet.

T5-small has been used for automating code-related tasks, including automatic bug fixing, code summarization, and assertion generation [96]. The pretrained T5 base model

was further pretrained using the CodeSearchNet dataset comprising 2.67 million instances, including raw source code, abstracted code, and technical natural language data. The fine-tuning phase involved multi-task learning across specialized datasets for different tasks, including automatic bug fixing, mutant generation, assertion generation, and code summarization. The fine-tuning was conducted on a mixture of tasks rather than each task individually to mitigate the risk of overfitting on smaller datasets. Hyperparameter tuning was performed with different learning rate strategies, with the Slanted Triangular Learning Rate with the best overall performance. Training was computationally expensive as it required approximately 343 hours on a Google Colab instance with 8 TPU cores and 35.5GB of RAM. The model's performance was evaluated using various metrics tailored to different tasks. Accuracy@K was used for bug-fixing and assertion generation, BLEU score for evaluating code summarization and mutant generation which assesses the similarity between the predicted and reference texts based on n-gram overlap. ROUGE-L was also used for code summarization that measures recall, precision, and F1-score based on the longest common subsequence between generated and reference summaries. Additionally, inference time was analyzed to determine the efficiency of the model across different beam search sizes. The model outperformed state-of-the-art approaches, particularly in mutant generation and assertion generation on raw code, which shows that the multi-task learning paradigm allows for improved generalization across different software engineering problems, as the model can handle diverse inputs.

Lemur model is an openly accessible language model optimized for both natural language and code-related tasks [99]. It builds upon the Llama 2 70B model as its base and undergoes a two-stage adaptation process: continual pretraining and instruction fine-tuning. The pretraining phase enhances the model's coding capabilities while preserving its natural language reasoning abilities. This is achieved by training it on a corpus comprising 90 billion tokens with a code-to-text ratio of 10:1, primarily sourced from The Stack dataset. The focus on scripting and interpreted languages (Python, SQL, Bash, etc.) enables dynamic interactions in agent-based scenarios. Additionally, text-based data from sources such as RefinedWeb, Wikipedia, ArXiv, and StackExchange contribute to maintaining the model's linguistic competence. Following pretraining, Lemur undergoes instruction fine-tuning to develop Lemur-Chat, an instruction-following variant of the model. This phase uses approximately 300,000 examples drawn from multiple datasets, including Open Assistant, OpenOrca, ShareGPT, ChatLogs, and Evol-CodeAlpaca. The fine-tuning process employs supervised instruction tuning using human-written and GPT-generated responses. The model is trained for two epochs using Hugging Face Transformers and the Accelerate Library on TPUv4-512 pods for efficient optimization. The evaluation of Lemur and Lemur-Chat is conducted across eight textual and coding benchmarks, including MMLU,

GSM8K, HumanEval, MBPP, and MultiPL-E. Furthermore, the model's capabilities in agent-based tasks are validated using environments such as WebArena and ALFWorld, where it demonstrates strong and competitive performance among open-source models. The fine-tuned Lemur-Chat model outperforms existing models like Llama 2 70B, StarCoder, and CodeLlama 34B.

WaveCoder is a family of code-centric LLMs developed using instruction tuning to enhance performance across multiple code-related tasks [100]. The model builds upon StarCoder-15B, CodeLlama 7B/13B, and DeepseekCoder-6.7B, and fine-tunes them on CodeSeaXDataset, a dataset containing 19,915 high-quality instruction instances across four primary tasks: code summarization, code generation, code translation, and code repair. Unlike prior works that rely on LLM-generated synthetic instructions (e.g., CodeAlpaca, WizardCoder), WaveCoder generates diverse instruction data by leveraging a LLM-based Generator-Discriminator framework combined with manual filtering and clustering (KCenterGreedy algorithm) to extract high-quality raw code samples from CodeSearchNet (2M+ examples, 4TB raw data). This enhances the diversity and reliability of instruction tuning data, enabling stronger generalization across various code tasks. The fine-tuning process employs supervised instruction tuning, where GPT-4 is used to refine 20K instructions in CodeSeaXDataset to improve response quality. Models are trained for three epochs using NVIDIA A100-80GB GPUs, employing tensor parallelism for StarCoder-15B and CodeLlama, while DeepseekCoder-6.7B is fine-tuned using Fully Sharded Data Parallel. Evaluation is conducted on HumanEval, MBPP, HumanEvalPack, HumanEvalFix, and HumanEvalExplain benchmarks, measuring pass@1, which assesses correctness across different programming tasks. WaveCoder-Pro-6.7B achieves 72.0% pass@1 on HumanEval, outperforming prior open-source models like Magicoder-DS (66.5%) and DeepseekCoder (49.4%), demonstrating the effectiveness of high-quality, diverse instruction tuning. The WaveCoder-Ultra-6.7B variant, fine-tuned on 130K instruction samples, establishes state-of-the-art generalization on multi-task code-related benchmarks among open-source models, highlighting the significance of diverse, instruction-tuned datasets for improving code LLM performance.

Although the models in this domain tackle similar tasks such as code generation, debugging, and summarization, they diverge in fine-tuning strategies based on task format, data type, and generalization goals. WizardCoder applies Evol-Instruct to strengthen instruction-following in code tasks, while StarCoder leverages Fill-in-the-Middle training to improve code completion capabilities in real-world scenarios where developers insert code into existing blocks. CodeT5+ uses span corruption as a pretraining objective because it aligns with T5's architecture and helps learn both semantic and syntactic structures by masking contiguous spans. This generalizes well across multiple tasks like translation or summarization. Additionally, OctoPack

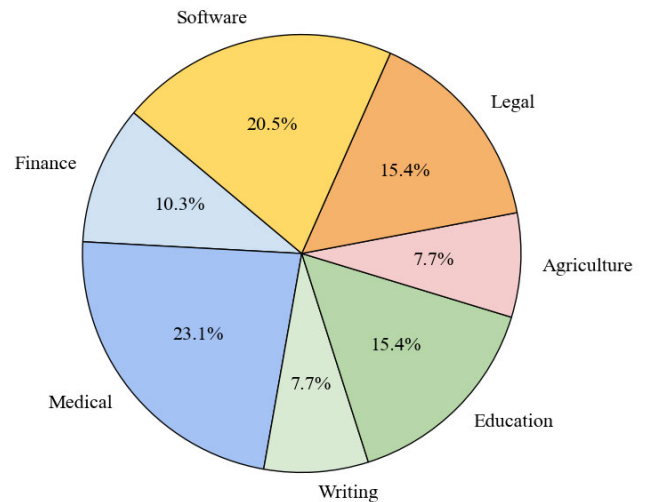


FIGURE 4. Distribution of different domains that apply LLMs.

aligns commit messages with code edits to simulate real-world programming workflows and documentation. Models like Llama-Reviewer and WaveCoder also adopt parameter-efficient tuning and diverse instruction datasets to optimize for performance with limited resources for structured feedback tasks like code review, where only limited labeled data is available. These variations highlight the importance of aligning fine-tuning techniques with specific downstream objectives, such as interactive debugging versus static code completion.

IV. RESEARCH FINDINGS AND GAPS

The previous section III has provided several different fine-tuned LLMs of diverse domains, their proposed method, and the involving challenges. This section will summarize and review the findings and research gaps in the existing literature to explore future improvements.

Fine-tuning methods over domain-specific LLMs open up different design choices that are influenced not only by task requirements but also by domain-specific requirements such as data shape and format, real-time requirements, user expectations, and safety considerations. For example, in the financial domain, models like FinGPT and Instruct-FinGPT leverage reinforcement learning and instruction tuning to adapt to evolving markets and reduce the reliance on costly manual annotation. Medical models tend to favor instruction tuning or supervised fine-tuning on curated clinical conversations and question answering datasets, balancing accuracy with safety and guideline adherence. For the legal field, supervised and instruction tuning approaches dominate, often with knowledge injection or retrieval modules to ensure interpretability and alignment with legal sources. Writing-focused LLMs, such as CoPoet and CoEdit, emphasize instruction tuning in combination with human judgment

and iterative editing pipelines to enable user control and creativity. Educational LLMs employ multi-stage fine-tuning with student-teacher distillation to encourage reasoning, personalization, and alignment with educational goals. For agriculture, approaches range from ontology-based fine-tuning (AgriBERT) to RAG (ShizishanGPT) to adversarial and reasoning-focused tuning (IPM-AgriGPT), reflecting the industry focus on scientific foundation, expert-level precision, and multilingual tool readiness. Programming-specific models (e.g., WizardCoder, StarCoder, OctoPack) typically integrate multi-turn instruction tuning, code-specific pre-training, and preference-based reinforcement learning to refine code generation, debugging, and compliance with developer intent.

These differences demonstrate that even for identical tasks e.g., question answering, sentiment analysis, or code completion, various domains require various fine-tuning strategies. The choice of method depends on the nature of the domain knowledge (structured or unstructured), availability of annotated data, criticality of output correctness, and end-user expectations. This highlights a key research finding: effective fine-tuning is not task-only, but domain-contextual, and requires synergy between tuning approach, model design, and application-specific constraints. Cross-domain transfer experiments and standard evaluation frameworks can help future research to better comprehend such trade-offs.

Our survey also shows that the application of LLMs is dominantly popular in medical applications and software development, while agriculture and writing have been given comparatively less attention as shown in Figure 4. Additionally, among the trends is the dominance of transformer-based architectures, particularly BERT and its variants (e.g., BioBERT, FinBERT, SciBERT). These models benefit from their ability to understand bidirectional context and thus are a requirement for in-domain tasks, such as named entity recognition and sentiment analysis. Llama-based models (e.g., Med42-v2, ChatDoctor, ChatLaw, LawLLM) and T-based models (e.g., SciFive, CoPoet, CodeT5) appear frequently across the surveyed applications. A potential rationale is that these models are open-weight and more accessible for fine-tuning. Additionally, they are available in various sizes, making them suitable for both academic and enterprise researchers who seek to adapt LLMs to specific tasks without relying on proprietary APIs or incurring the high computational costs. GPT-based models (e.g., ShizishanGPT, and FinGPT) also remain popular due to their generative nature, which makes them suitable for tasks such as report generation and conversational AI.

The survey also highlights the shift from basic supervised fine-tuning to more advanced strategies, such as instruction tuning. Early domain-specific LLMs like FinBERT, BioBERT, and LegalBERT employed classic supervised fine-tuning techniques. Such methods involved optimizing the model parameters on task datasets via loss functions like cross-entropy, which adapted the models for classification and extraction tasks. However, they often lacked

generalization in zero-shot or multi-task settings. More recent models, such as ChatDoctor, ChatLaw, and LawGPT, are trained on (instruction, output) pairs to follow human instructions and generalize well on diverse tasks. This has been effective at enhancing controllability, alignment with user expectations, and performance on unseen instructions, multitask, and conversational capabilities. Furthermore, RAG models are being utilized widely as they improve the factuality of LLMs using external structured knowledge. Apart from that, with the growth in the size of LLMs, it becomes increasingly expensive to update all the parameters in fine-tuning. Despite the theoretical and practical limitations discussed in Section II, these techniques alongside parameter-efficient methods like LoRA, prefix tuning, and 4-bit quantized tuning continue to be widely adopted for fine-tuning purposes. This is because of demonstrated empirical effectiveness, ability to generalize to low-resource environments, and reduced computational demands compared to full fine-tuning.

Across the surveyed models, F1-score, precision, recall, and BLEU are the most common reported evaluation metrics. These metrics highlight model strengths in specific tasks, but there is often a lack of standardized benchmarks, which makes cross-model comparisons challenging. For example, while FinBERT has high F1-scores for financial sentiment analysis, AUC-ROC scores for generic financial prediction tasks are less reported. Similarly, creative models like CoPoet rely on human evaluation for creativity and coherence, which is less objective than automatic metrics.

Performance is significantly affected by the choice of base model. For example, BERT-based models excel at tasks requiring understanding of compact text data, while GPT-based models are more appropriate for generative and conversational tasks. Dataset selection is equally crucial; domain-specific, high-quality datasets such as PubMed (BioBERT) and FiQA (FinBERT) enable models to achieve beyond their general-purpose counterparts.

Conversely, limited or specifically targeted datasets, such as the small question answering dataset for ShizishanGPT, restrict model performance and generalizability. Aside from considerations in model and dataset selection, a number of systemic challenges continue to impede the successful implementation of fine-tuned LLMs:

- **Domain Representation and Applicability:** Special domain fine-tuned LLMs have proven to be useful in specialized domains like healthcare, finance, education, legal systems, and software development for named entity recognition, sentiment analysis, writing tasks, and code generation. Despite these advances, underrepresented domains such as law enforcement, public safety, cybersecurity, and criminology lack specialized fine-tuned models. These domains possess some special challenges, such as real-time decision-making, multilingual support, and ethical standards. Moreover, most domains face the problem of limited dataset availability. For instance, WizardCoder

leverages synthetic instruction data (Evol-Instruct) to address dataset scarcity, but its generalizability to the real world is relatively unexplored given the lack of human-curated or multilingual datasets. Our survey also reflects that while domains like programming and finance have more established benchmarks, domains like education and agriculture utilize smaller or tailor-made benchmarks. To bridge this gap, researchers must prioritize model and benchmark development for low-resource domains and multilingual and culturally sensitive applications. Government agencies and organizations can be partnered with to enable the development of anonymized datasets for these sensitive domains.

- **Dataset Quality and Availability:** Specialized, high-quality datasets such as MIMIC-III, PubMed, and FiQA have been utilized to achieve success in fine-tuning LLMs in the aforementioned domains. These datasets provide domain-specific knowledge that is rich and allows the models to outperform their general-purpose counterparts. However, dataset limitations remain a significant bottleneck, particularly in new or underserved domains. For instance, legal datasets are geographically specific and limit model generalizability. Concerns of privacy in areas like law enforcement and medicine further restrict the availability of full datasets. Future efforts must be put into creating standardized, publicly available datasets that reflect diverse regions and cultures while preserving data privacy. Synthetic data generation can also provide gaps in areas where there is sparse labeled data, such as rare medical conditions or legal systems in resource-constrained settings.
- **Computational Resource Constraints:** Large models like GPT-4 and specific variants of Llama and T5 have set new performance standards, but their high computational demands decrease accessibility. Domain-specific fine-tuned models can mitigate this issue, but they still have deployment challenges in resource-constrained environments. Computational inefficiency of current architectures hinders their use in real-time or edge applications, where low latency and efficient resource usage are crucial. This is particularly demanding for applications that require quick decision-making, e.g., financial trading or public safety. Techniques like model pruning and knowledge distillation can be leveraged to reduce the computational overhead without affecting performance. Adaptive learning techniques could further provide efficiency by allocating computational resources to task-relevant data.
- **Explainability and Ethical Considerations:** Explainability is a growing need in many applications like healthcare and law, where trust and accountability are important. However, most fine-tuned LLMs are still limited in explaining their decisions. Models like CoPoet, for example, are useful for collaborative writing, yet they are not clear about their stylistic choices, and StarCoder is unable to properly explain its debug suggestions, which can become problematic in professional settings. Similarly, LawLLM shows promise in tasks like legal document classification and case retrieval, but it lacks transparency in its reasoning process, so it will be difficult for legal professionals to fully trust or audit its predictions, especially in high-stakes legal decisions. Utilizing explainability methods like feature attribution and decision visualization can make the model more trustworthy. Lack of explainability, along with ethical concerns like bias and the misuse potential, holds back the deployment of fine-tuned models in applications that require more transparency. For example, in LLM-based medical systems, an incorrect diagnosis or treatment without clear reasoning could lead to harmful outcomes. Moreover, models trained on patient records may expose sensitive health information. This holds special relevance with the popularity of LLMs in the healthcare sector, as revealed in Figure 4. Therefore, ethical considerations must also be a top priority for researchers through the development of bias-reduction methods and compliance with regulatory requirements for data privacy and sensitivity.
- **Standardization and Benchmarking:** The success of fine-tuned models is typically evaluated using task-specific metrics, but a proliferation of state-of-the-art methods makes comparison between models difficult. Existing benchmarks are typically domain-specific and lack a shared evaluation framework. For instance, LawGPT evaluates its performance using custom legal exams and few-shot tasks on LegalBench, while other legal LLMs like ChatLaw or Legal-BERT use different, often unpublished metrics. Similarly, in creative writing, models like CoPoet and CoEdit are evaluated primarily by human judgment of creativity and coherence without reproducible and automatic evaluation. Moreover, evaluation metrics like ROUGE can introduce bias for comparing extractive and abstractive approaches. As observed in [101], ROUGE tends to underestimate semantically accurate but lexically diverse outputs typical of LLMs, especially in BERT-based summarization. This incomparability frustrates the identification of best practices, frustrates the progress in advancing methodologies, and illustrates the need for more robust, semantically-aware metrics of evaluation for comparing fine-tuned LLMs across domains. Standardized benchmarks and cross-domain evaluation frameworks are important to advance the field, as they can provide a comprehensive view of model performance and improve reproducibility as well.
- **Multimodal Capabilities:** Most domain-specific LLMs primarily process textual data, yet many specialized applications, such as radiology, legal case analysis, and financial market forecasting, require multimodal reasoning across text, images, and structured tabular data. While some efforts integrate multimodal capabilities (e.g., Me-Llama for medical images), these remain

narrow in scope and generalizability. Future research would have to explore cross-modal training techniques and lightweight multimodal fusion architectures for greater versatility in real-world applications.

V. THREATS TO VALIDITY

While we have carefully chosen and analyzed a wide range of LLMs in this survey, there are some threats to validity to consider.

The field of LLMs is rapidly evolving, and more recent models may be developed and released after the completion of this survey. These models can outperform the ones included in our analysis in terms of accuracy, robustness, efficiency, or other domain-specific metrics. However, this progress does not invalidate the utility or relevance of the surveyed models. Many existing models continue to be widely used in real-world applications and as baselines for new frameworks and architectures. Future models may also demonstrate better balances between performance and cost, such as lower memory consumption or enhanced scalability. These improvements can be achieved by optimization methods or more efficient training and inference pipelines. While our survey acknowledges some of these developments where applicable, ongoing advances may further change the computational landscape as well.

It is noteworthy to mention that performance comparisons across models are based on benchmarks availability. These benchmarks may become outdated or replaced by more comprehensive evaluation methods. Consequently, model rankings and performance may shift as other domain-aligned evaluation protocols are designed.

In addition, some of the models surveyed are trained on proprietary or unpublished datasets, making reproducibility and cross-model comparisons challenging. This can affect the generalization of certain findings, particularly where datasets are tailored to specific regions, institutions, or tasks.

VI. CONCLUSION

This survey presents a comprehensive overview of fine-tuning domain-specific LLMs and how general-purpose models such as GPT-3, GPT-4, BERT, T5, and Llama are fine-tuned for specialized use in finance, medicine, law, education, agriculture, software development, and creative writing and have transformed the field of NLP. While not providing a comprehensive examination of individual papers, this survey serves as a starting point for researchers and developers looking to leverage LLMs in a specific use case. By combining state-of-the-art research and focusing on successes and shortcomings, we aimed to inspire further advancement in applying LLMs to real-world applications. Through detailed examination of over 30 fine-tuned models, we have underscored both the benefits and challenges associated with fine-tuning. We identified common techniques, including instruction tuning, domain-adaptive pretraining, knowledge integration, and parameter-efficient techniques such as LoRA, and evaluated their impact on task performance, data requirements, and computational expense,

and demonstrated the potential to improve LLM capabilities while addressing practical limitations.

Our comparative findings show that there is no fine-tuning method that fits all models and scenarios. Instead, the choices are often guided by domain constraints, model size, and data modality. Additionally, instruction tuning is prevalent in high-resource settings, while prompt engineering and adaptation dominate where fine-tuning is impractical. Finally, performance gains are frequently domain-specific, with limited evidence of cross-domain generalization. Moreover, many models show promising results in academic benchmarks but lack real-world validation. There is limited work evaluating the practical deployment of models in production systems or integrating them with existing workflows and policies.

Therefore, future work should address standardizing metrics and benchmarks for assessing fine-tuned model performance across diverse applications and address practical fine-tuning and deployment issues of LLMs. This also could include a comparison of fine-tuning methods based on computational cost and performance trade-offs. By defining standard benchmarks and evaluation frameworks, we can leverage LLMs to meet user needs and real-world demands more effectively. In addition, there is still a gap in adapting LLMs in domains such as environmental science, transportation, law enforcement, and other critical applications, such as public safety and emergency response, where improving processing workflows by speeding up such workflows with high-end NLP can be beneficial. It is recommended to explore the applicability of the fine-tuning techniques to these areas in future work.

Finally, future research should also investigate how LLMs can be fine-tuned to meet ethical needs for trustworthiness, transparency, and data privacy concerns. As discussed earlier, fine-tuned LLMs can still pose a risk of misuse or demonstrate domain-specific biases that limit their safe use in decision-making systems. Addressing these concerns through robust evaluation, bias mitigation strategies, and transparent workflows is essential for the responsible use of LLMs across all domains.

REFERENCES

- [1] Y. Liu, T. Han, S. Ma, J. Zhang, Y. Yang, J. Tian, H. He, A. Li, M. He, Z. Liu, Z. Wu, L. Zhao, D. Zhu, X. Li, N. Qiang, D. Shen, T. Liu, and B. Ge, "Summary of ChatGPT-related research and perspective towards the future of large language models," *Meta-Radiol.*, vol. 1, no. 2, Sep. 2023, Art. no. 100017.
- [2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [3] T. B. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
- [4] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [5] C. Zhou, Q. Li, C. Li, J. Yu, Y. Liu, G. Wang, K. Zhang, C. Ji, Q. Yan, L. He, H. Peng, J. Li, J. Wu, Z. Liu, P. Xie, C. Xiong, J. Pei, P. S. Yu, and L. Sun, "A comprehensive survey on pretrained foundation models: A history from BERT to ChatGPT," *Int. J. Mach. Learn. Cybern.*, vol. 16, no. 12, pp. 1–65, Dec. 2025.

- [6] B. Zhang, H. Yang, and X.-Y. Liu, "Instruct-FinGPT: Financial sentiment analysis by instruction tuning of general-purpose large language models," 2023, *arXiv:2306.12659*.
- [7] S. Zhang, L. Dong, X. Li, S. Zhang, X. Sun, S. Wang, J. Li, R. Hu, T. Zhang, F. Wu, and G. Wang, "Instruction tuning for large language models: A survey," 2023, *arXiv:2308.10792*.
- [8] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. Wei Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," 2021, *arXiv:2109.01652*.
- [9] A. Gudibande, E. Wallace, C. Snell, X. Geng, H. Liu, P. Abbeel, S. Levine, and D. Song, "The false promise of imitating proprietary LLMs," 2023, *arXiv:2305.15717*.
- [10] J. Zheng, H. Hong, F. Liu, X. Wang, J. Su, Y. Liang, and S. Wu, "Fine-tuning large language models for domain-specific machine translation," 2024, *arXiv:2402.15061*.
- [11] J. Sun, C. Mei, L. Wei, K. Zheng, N. Liu, M. Cui, and T. Li, "Dial-insight: Fine-tuning large language models with high-quality domain-specific data preventing capability collapse," 2024, *arXiv:2403.09167*.
- [12] A. Wang, C. Liu, J. Yang, and C. Weng, "Fine-tuning large language models for rare disease concept normalization," *J. Amer. Med. Inform. Assoc.*, vol. 31, no. 9, pp. 2076–2083, Sep. 2024.
- [13] B. Alt, U. Keßner, A. Taranovic, D. Katic, A. Hermann, R. Jäkel, and G. Neumann, "Domain-specific fine-tuning of large language models for interactive robot programming," in *European Robotics Forum*. Cham, Switzerland: Springer, 2024, pp. 274–279.
- [14] W. Lu, R. K. Luu, and M. J. Buehler, "Fine-tuning large language models for domain adaptation: Exploration of training strategies, scaling, model merging and synergistic capabilities," 2024, *arXiv:2409.03444*.
- [15] E. Radiya-Dixit and X. Wang, "How fine can fine-tuning be? Learning efficient language models," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2020, pp. 2435–2443.
- [16] S. Gupta, U. Deshpande, T. Janssen, and S. Sundaraman, "Symbiosis: Multi-adapter inference and fine-tuning," 2025, *arXiv:2507.03220*.
- [17] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," 2021, *arXiv:2106.09685*.
- [18] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. D. Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2790–2799.
- [19] Z. Lin, A. Madotto, and P. Fung, "Exploring versatile generative language model via parameter-efficient transfer learning," 2020, *arXiv:2004.03829*.
- [20] A. Rücklé, G. Geigle, M. Glockner, T. Beck, J. Pfeiffer, N. Reimers, and I. Gurevych, "AdapterDrop: On the efficiency of adapters in transformers," 2020, *arXiv:2010.11918*.
- [21] J. Pfeiffer, A. Kamath, A. Rücklé, K. Cho, and I. Gurevych, "Adapter-Fusion: Non-destructive task composition for transfer learning," 2020, *arXiv:2005.00247*.
- [22] X. Lisa Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," 2021, *arXiv:2101.00190*.
- [23] J. Y.-C. Hu, M. Su, E.-J. Kuo, Z. Song, and H. Liu, "Computational limits of low-rank adaptation (LoRA) fine-tuning for transformer models," in *Proc. ICLR Workshop Deep Generative Model Mach. Learn. Theory*, 2025, pp. 1–38.
- [24] R. Shuttleworth, J. Andreas, A. Torralba, and P. Sharma, "LoRA vs full fine-tuning: An illusion of equivalence," 2024, *arXiv:2410.21228*.
- [25] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, "GOBO: Quantizing attention-based NLP models for low latency and energy efficient inference," in *Proc. 53rd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2020, pp. 811–824.
- [26] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "GPT3.int8(): 8-bit matrix multiplication for transformers at scale," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 30318–30332.
- [27] A. Ardakani, A. Haan, S. Tan, D. Thom Popovici, A. Cheung, C. Iancu, and K. Sen, "SlimFit: Memory-efficient fine-tuning of transformer-based models using training dynamics," 2023, *arXiv:2305.18513*.
- [28] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2025, pp. 5998–6008.
- [29] G. Wenzek, M.-A. Lachaux, A. Conneau, V. Chaudhary, F. Guzmán, A. Joulin, and E. Grave, "CCNet: Extracting high quality monolingual datasets from Web crawl data," 2019, *arXiv:1911.00359*.
- [30] J. Achiam, "Gpt-4 technical report," 2023, *arXiv:2303.08774*.
- [31] J. D. M.-W. C. Kenton and L. K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, vol. 1, no. 2, pp. 4171–4186, doi: [10.48550/arXiv.1810.04805](https://doi.org/10.48550/arXiv.1810.04805).
- [32] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019, *arXiv:1910.01108*.
- [33] C. Raffel, N. Shazeer, A. P. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2019.
- [34] H. W. Chung et al., "Scaling instruction-finetuned language models," *J. Mach. Learn. Res.*, vol. 25, no. 70, pp. 1–53, 2022.
- [35] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, S. Presser, and C. Leahy, "The pile: An 800GB dataset of diverse text for language modeling," 2021, *arXiv:2101.00027*.
- [36] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, (2023). *Alpaca: A Strong, Replicable Instruction-Following Model*. [Online]. Available: <https://crfm.stanford.edu/2023/03/13/alpaca.html>
- [37] W.-L. Chiang et al. (2023). *Vicuna: An Open-Source Chatbot Impressing GPT-4 With 90%* ChatGPT Quality*. Accessed: Apr. 14, 2023. [Online]. Available: <https://vicuna.lmsys.org>
- [38] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," 2019, *arXiv:1908.10063*.
- [39] P. Malo, A. Sinha, P. Korhonen, J. Wallenius, and P. Takala, "Good debt or bad debt: Detecting semantic orientations in economic texts," *J. Assoc. Inf. Sci. Technol.*, vol. 65, no. 4, pp. 782–796, Apr. 2014.
- [40] M. Maia, S. Handschuh, A. Freitas, B. Davis, R. McDermott, M. Zarrouk, and A. Balahur, "WWW'18 open challenge: Financial opinion mining and question answering," in *Proc. Companion Web Conf. Web Conf. WWW*, 2018, pp. 1941–1942.
- [41] Q. Xie, W. Han, X. Zhang, Y. Lai, M. Peng, A. Lopez-Lira, and J. Huang, "PIXIU: A large language model, instruction data and evaluation benchmark for finance," 2023, *arXiv:2306.05443*.
- [42] X.-Y. Liu, G. Wang, H. Yang, and D. Zha, "FinGPT: Democratizing internet-scale data for financial large language models," 2023, *arXiv:2307.10485*.
- [43] Z. Du, Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, and J. Tang, "GLM: General language model pretraining with autoregressive blank infilling," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics*, 2022, pp. 320–335.
- [44] OpenAI. *ChatGPT*. Accessed: Jan. 3, 2025. [Online]. Available: <https://chat.openai.com>
- [45] Y. Luo, Z. Yang, F. Meng, Y. Li, J. Zhou, and Y. Zhang, "An empirical study of catastrophic forgetting in large language models during continual fine-tuning," 2023, *arXiv:2308.08747*.
- [46] J. Lee, W. Yoon, S. Kim, D. Kim, S. Kim, C. H. So, and J. Kang, "BioBERT: A pre-trained biomedical language representation model for biomedical text mining," *Bioinformatics*, vol. 36, no. 4, pp. 1234–1240, Feb. 2020.
- [47] L. N. Phan, J. T. Anibal, H. Tran, S. Chanana, E. Bahadroglu, A. Peltekian, and G. Altan-Bonnet, "SciFive: A text-to-text transformer model for biomedical literature," 2021, *arXiv:2106.03598*.
- [48] H. Xiong, S. Wang, Y. Zhu, Z. Zhao, Y. Liu, L. Huang, Q. Wang, and D. Shen, "DoctorGLM: Fine-tuning your Chinese doctor is not a herculean task," 2023, *arXiv:2304.01097*.
- [49] Z. Liu et al., "Radiology-llama2: Best-in-class large language model for radiology," 2023, *arXiv:2309.06419*.
- [50] A. E. W. Johnson, T. J. Pollard, S. J. Berkowitz, N. R. Greenbaum, M. P. Lungren, C.-Y. Deng, R. G. Mark, and S. Horng, "MIMIC-CXR, a de-identified publicly available database of chest radiographs with free-text reports," *Sci. Data*, vol. 6, no. 1, p. 317, Dec. 2019.
- [51] C. Christophe, P. K. Kanithi, T. Raha, S. Khan, and M. A. Pimentel, "Med42-v2: A suite of clinical LLMs," 2024, *arXiv:2408.06142*.
- [52] Y. Li, Z. Li, K. Zhang, R. Dan, S. Jiang, and Y. Zhang, "ChatDoctor: A medical chat model fine-tuned on a large language model meta-AI (LLaMA) using medical domain knowledge," *Cureus*, vol. 15, pp. 1–12, Jun. 2023, doi: [10.7759/cureus.40895](https://doi.org/10.7759/cureus.40895).
- [53] *Health Care Magic*. Accessed: Dec. 2025. [Online]. Available: <https://www.healthcaremagic.com>

- [54] J. Chen, X. Wang, K. Ji, A. Gao, F. Jiang, S. Chen, H. Zhang, D. Song, W. Xie, C. Kong, J. Li, X. Wan, H. Li, and B. Wang, "HuatuoGPT-II, one-stage training for medical adaption of LLMs," 2023, *arXiv:2311.09774*.
- [55] Q. Xie, Q. Chen, A. Chen, P. Cheng, Y. Hu, F. Lin, X. Peng, J. Huang, J. Zhang, V. Keloth, X. Zhou, H. He, L. Ohno-Machado, Y. Wu, H. Xu, and J. Bian, "Me-LLaMA: Foundation large language models for medical applications," *Res. Square*, vol. 3, May 2024, doi: [10.21203/rs.3.rs-4240043/v1](https://doi.org/10.21203/rs.3.rs-4240043/v1).
- [56] D. Rossi, A. A. Citarella, F. De Marco, L. Di Biasi, and G. Tortora, "Comparative analysis of diabetes diagnosis: WE-LSTM networks and WizardLM-powered DiabeTalk chatbot," in *Proc. IEEE Int. Conf. Bioinf. Biomed. (BIBM)*, Dec. 2024, pp. 6859–6866.
- [57] C. Xu, Q. Sun, K. Zheng, X. Geng, P. Zhao, J. Feng, C. Tao, D. Jiang, and J. Daxin, "WizardLM: Empowering large pre-trained language models to follow complex instructions," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024, pp. 1–22.
- [58] T. Chakrabarty, V. Padmakumar, and H. He, "Help me write a poem: Instruction tuning as a vehicle for collaborative poetry writing," 2022, *arXiv:2210.13669*.
- [59] T. Chakrabarty, A. Saakyan, and S. Muresan, "Don't go far off: An empirical study on neural poetry translation," 2021, *arXiv:2109.02972*.
- [60] V. Raheja, D. Kumar, R. Koo, and D. Kang, "CoEdit: Text editing by task-specific instruction tuning," 2023, *arXiv:2305.09857*.
- [61] Z. Myung Kim, W. Du, V. Raheja, D. Kumar, and D. Kang, "Improving iterative text revision by learning where to edit from other revision tasks," 2022, *arXiv:2212.01350*.
- [62] Z. Xu, S. Cai, M. Varma T, S. Venugopalan, and S. Zhai, "SkipWriter: LLM-powered abbreviated writing on tablets," in *Proc. 37th Annu. ACM Symp. User Interface Softw. Technol.*, Oct. 2024, pp. 1–13.
- [63] R. Anil et al., "PaLM 2 technical report," 2023, *arXiv:2305.10403*.
- [64] G. Srivastav, S. Kant, D. Srivastava, N. Sharma, and Y.-C. Hu, "An efficient sentiment analysis technique based on fine-tuned EdBERT for virtual learning environments," *Multimedia Tools Appl.*, vol. 84, no. 18, pp. 1–15, Jul. 2024.
- [65] Z. Liu, X. He, L. Liu, T. Liu, and X. Zhai, "Context matters: A strategy to pre-train language model for science education," in *Proc. Int. Conf. Artif. Intell. Educ.*, 2023, pp. 666–674.
- [66] E. Latif, G.-G. Lee, K. Neumann, T. Kastorff, and X. Zhai, "G-SciEdBERT: A contextualized LLM for science assessment tasks in German," 2024, *arXiv:2402.06584*.
- [67] *PISA 2015 Results (Volume I): Excellence and Equity in Education*, PISA, OECD Publishing, Paris, France, 2016, doi: [10.1787/9789264266490-en](https://doi.org/10.1787/9789264266490-en).
- [68] *PISA 2018 Results (Volume I): What Students Know and Can Do*, OECD Publishing, Paris, France, 2019, doi: [10.1787/5f07c754-en](https://doi.org/10.1787/5f07c754-en).
- [69] E. Latif and X. Zhai, "Fine-tuning ChatGPT for automatic scoring," *Comput. Educ., Artif. Intell.*, vol. 6, Jun. 2024, Art. no. 100210.
- [70] Y. Jeong and E. Kim, "SciDeBERTa: Learning DeBERTa for science technology documents and fine-tuning information extraction tasks," *IEEE Access*, vol. 10, pp. 60805–60813, 2022.
- [71] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with disentangled attention," 2020, *arXiv:2006.03654*.
- [72] Y. Luan, L. He, M. Ostendorf, and H. Hajishirzi, "Multi-task identification of entities, relations, and coreference for scientific knowledge graph construction," 2018, *arXiv:1808.09602*.
- [73] J.-D. Kim, T. Ohta, Y. Tateisi, and J. Tsujii, "Genia corpus—A semantically annotated corpus for bio-textmining," *Bioinformatics*, vol. 19, pp. 180–182, Jan. 2003.
- [74] S. Rezayi, Z. Liu, Z. Wu, C. Dhakal, B. Ge, C. Zhen, T. Liu, and S. Li, "AgriBERT: Knowledge-infused agricultural language models for matching food and nutrition," in *Proc. 31st Int. Joint Conf. Artif. Intell.*, Jul. 2022, pp. 5150–5156.
- [75] S. Yang, Z. Liu, and W. Mayer, "ShizishanGPT: An agricultural large language model integrating tools and resources," in *Proc. Int. Conf. Web Inf. Syst. Eng.*, 2024, pp. 284–298.
- [76] Y. Zhang, Q. Fan, X. Chen, M. Li, Z. Zhao, F. Li, and L. Guo, "IPM-AgriGPT: A large language model for pest and disease management with a G-EA framework and agricultural contextual reasoning," *Mathematics*, vol. 13, no. 4, p. 566, Feb. 2025.
- [77] A. Zeng et al., "ChatGLM: A family of large language models from GLM-130B to GLM-4 all tools," 2024, *arXiv:2406.12793*.
- [78] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, "LEGAL-BERT: The muppets straight out of law school," 2020, *arXiv:2010.02559*.
- [79] I. Chalkidis, I. Androutsopoulos, and N. Aletras, "Neural legal judgment prediction in English," 2019, *arXiv:1906.02059*.
- [80] J. Cui, Z. Li, Y. Yan, B. Chen, and L. Yuan, "Chatlaw: Open-source legal large language model with integrated external knowledge bases," in *Proc. CoRR*, 2023, pp. 1–11, doi: [10.48550/arXiv.2306.16092](https://doi.org/10.48550/arXiv.2306.16092).
- [81] S. Yao, Q. Ke, Q. Wang, K. Li, and J. Hu, "Lawyer GPT: A legal large language model with enhanced domain knowledge and reasoning capabilities," in *Proc. 3rd Int. Symp. Robot., Artif. Intell. Inf. Eng.*, Jul. 2024, pp. 108–112.
- [82] Z. Zhou, J.-X. Shi, P.-X. Song, X.-W. Yang, Y.-X. Jin, L.-Z. Guo, and Y.-F. Li, "LawGPT: A Chinese legal knowledge-enhanced large language model," 2024, *arXiv:2406.04614*.
- [83] S. Yue, W. Chen, S. Wang, B. Li, C. Shen, S. Liu, Y. Zhou, Y. Xiao, S. Yun, X. Huang, and Z. Wei, "DISC-LawLLM: Fine-tuning large language models for intelligent legal services," 2023, *arXiv:2309.11325*.
- [84] D. Shu, H. Zhao, X. Liu, D. Demeter, M. Du, and Y. Zhang, "LawLLM: Law large language model for the U.S. legal system," in *Proc. 33rd ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2024, pp. 4882–4889.
- [85] G. Team et al., "Gemini: A family of highly capable multimodal models," 2023, *arXiv:2312.11805*.
- [86] (2024). *CaseLaw Project*. [Online]. Available: <https://case.law/>
- [87] R. Li et al., "StarCoder: May the source be with you!" 2023, *arXiv:2305.06161*.
- [88] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, W. Hu, C. Tao, J. Ma, Q. Lin, and D. Jiang, "WizardCoder: Empowering code large language models with evol-instruct," 2023, *arXiv:2306.08568*.
- [89] B. Rozière et al., "Code llama: Open foundation models for code," 2023, *arXiv:2308.12950*.
- [90] N. Muennighoff, Q. Liu, A. Zebaze, Q. Zheng, B. Hui, T. Yue Zhuo, S. Singh, X. Tang, L. von Werra, and S. Longpre, "OctoPack: Instruction tuning code large language models," 2023, *arXiv:2308.07124*.
- [91] J. Lu, L. Yu, X. Li, L. Yang, and C. Zuo, "LLaMA-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning," in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng. (ISSRE)*, Oct. 2023, pp. 647–658.
- [92] R. Tufano, S. Masiero, A. Mastropaolo, L. Pascarella, D. Poshyanyk, and G. Bavota, "Using pre-trained models to boost code review automation," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng. (ICSE)*, May 2022, pp. 2291–2302.
- [93] Z. Li, S. Lu, D. Guo, N. Duan, S. Jannu, G. Jenks, D. Majumder, J. Green, A. Svyatkovskiy, S. Fu, and N. Sundaresan, "Automating code review activities by large-scale pre-training," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2022, pp. 1035–1047.
- [94] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," 2021, *arXiv:2109.00859*.
- [95] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou, "CodeGen2: Lessons for training LLMs on programming and natural languages," 2023, *arXiv:2305.02309*.
- [96] A. Mastropaolo, S. Scalabrino, N. Cooper, D. N. Palacio, D. Poshyanyk, R. Oliveto, and G. Bavota, "Studying the usage of text-to-text transfer transformer to support code-related tasks," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, May 2021, pp. 336–347.
- [97] C. Watson, M. Tufano, K. Moran, G. Bavota, and D. Poshyanyk, "On learning meaningful assert statements for unit test cases," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. (ICSE)*, Oct. 2020, pp. 1398–1409.
- [98] S. Haque, A. LeClair, L. Wu, and C. McMillan, "Improved automatic summarization of subroutines via attention to file context," in *Proc. IEEE/ACM 17th Int. Conf. Mining Softw. Repositories (MSR)*, May 2020, pp. 300–310.
- [99] Y. Xu, H. Su, C. Xing, B. Mi, Q. Liu, W. Shi, B. Hui, F. Zhou, Y. Liu, T. Xie, Z. Cheng, S. Zhao, L. Kong, B. Wang, C. Xiong, and T. Yu, "Lemur: Harmonizing natural language and code for language agents," 2023, *arXiv:2310.06830*.
- [100] Z. Yu, X. Zhang, N. Shang, Y. Huang, C. Xu, Y. Zhao, W. Hu, and Q. Yin, "WaveCoder: Widespread and versatile enhancement for code large language models by instruction tuning," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 5140–5153.

- [101] A. A. Citarella, M. Barbella, M. G. Ciobanu, F. De Marco, L. Di Biasi, and G. Tortora, "Assessing the effectiveness of ROUGE as unbiased metric in extractive vs. Abstractive summarization techniques," *J. Comput. Sci.*, vol. 87, May 2025, Art. no. 102571.



PARIA SARZAEIM received the M.A.Sc. degree in electrical and computer engineering from Ontario Tech University, Canada, in 2024. She is currently an AI Researcher with Mobile Innovations Corporation, Canada. Her research interests include machine learning, generative artificial intelligence, and large language models.



GARY BAUER is the Founder and the CEO of Mobile Innovations Corporation. He is a pioneer of secure mobile computing, cloud adoption, and generative AI for public and private enterprise clients. Under his leadership, Mobile Innovations has developed, implemented, and supported software for law enforcement, transportation, healthcare, and public service in Canada, U.K., and USA. A graduate of the University of Toronto Aerospace program, he complements his 30 years of hands-on leadership with expertise in research and development and international business development.



AKRAMUL AZIM (Senior Member, IEEE) is currently an Associate Professor with the Department of Electrical, Computer, and Software Engineering and the Head of the Real-Time Embedded Software (RTEMSOFT) Research Group, Ontario Tech University, Oshawa, ON, Canada. His research interests include real-time systems, embedded software, software verification and validation, safety-critical software, and intelligent transportation systems. He is a Professional Engineer in Ontario.



MASOUD MAKREHCHI is currently an Associate Professor with the Department of Electrical, Computer, and Software Engineering, Ontario Tech University, Oshawa, ON, Canada. His research interests include natural language processing, large language models, and AI.

...